深圳大学
SHENZHEN UNIVERSITY

模式识别
Pattern Recognition

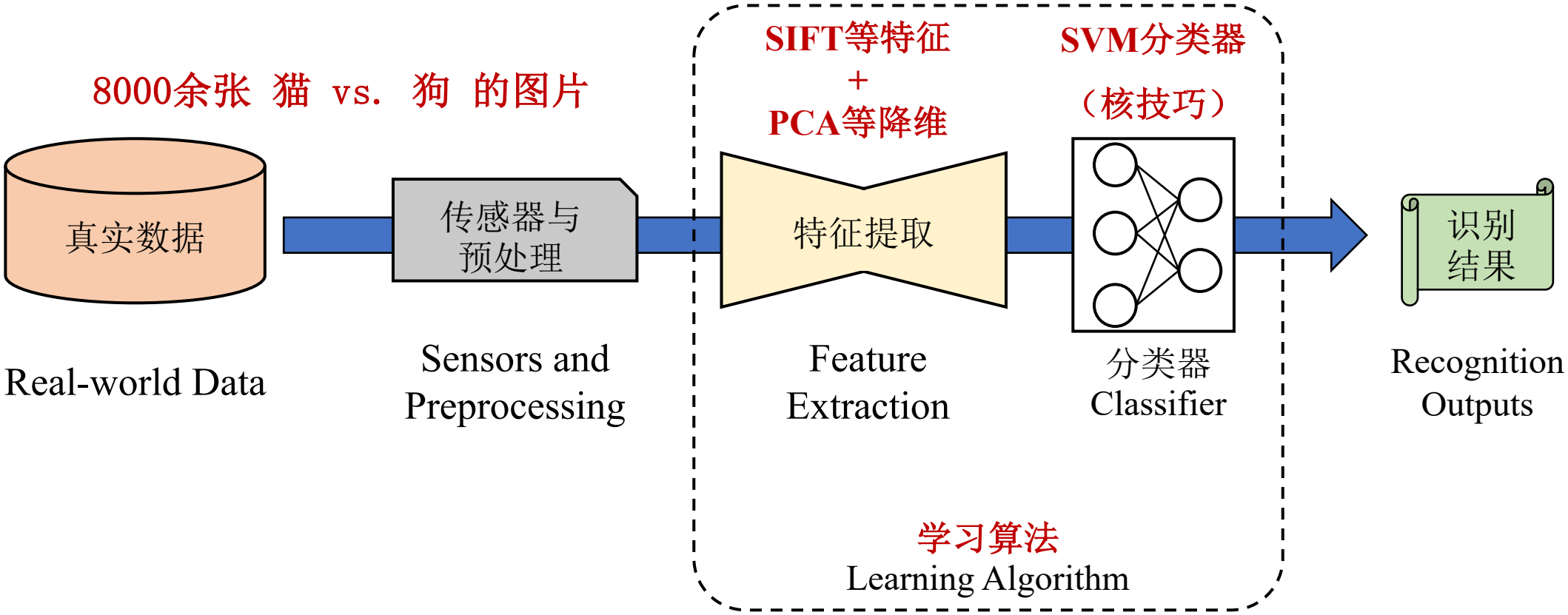
——与计算机视觉
and Computer Vision

樊超

大数据系统计算技术国家工程实验室@深圳大学

E-mail：chaofan996@szu.edu.cn

Where we are?

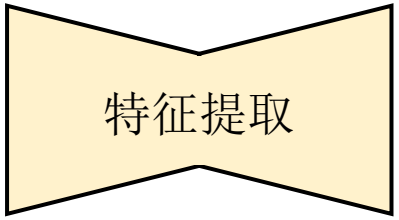

8000余张 猫 vs. 狗 的图片
真实数据
传感器与
预处理
SIFT等特征
+
PCA等降维
特征提取
SVM分类器
（核技巧）
分类器
识别
结果
Real-world Data
Sensors and
Preprocessing
Feature
Extraction
Classifier
Recognition
Outputs
学习算法
Learning Algorithm

Where we are?

SIFT等特征 $\boldsymbol{x}$

+

PCA等降维



Feature Extraction

$\mathbf{U}$ serves two functions:

- Encode: $\mathbf{z} = \mathbf{U}^\top \mathbf{x}, \quad z_j = \mathbf{u}_j^\top \mathbf{x}$
- Decode: $\tilde{\mathbf{x}} = \mathbf{U}\mathbf{z} = \sum_{j=1}^{k} z_j \mathbf{u}_j$

PCA objective 1: reconstruction error

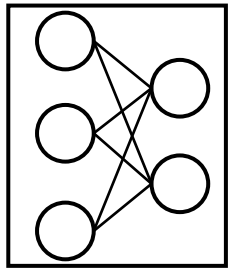
$$\min_{\mathbf{U} \in \mathbb{R}^{d \times k}} \sum_{i=1}^{n} \|\mathbf{x}_i - \mathbf{U}\mathbf{U}^\top \mathbf{x}_i\|^2$$

PCA objective 2: projected variance

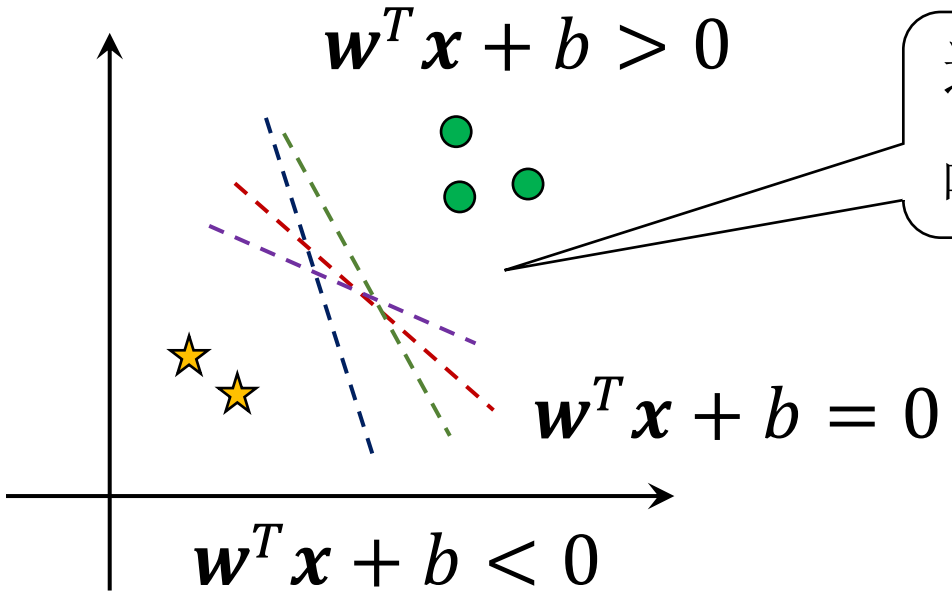
$$\max_{\mathbf{U} \in \mathbb{R}^{d \times k}, \mathbf{U}^\top \mathbf{U} = I} \hat{\mathbb{E}}[\|\mathbf{U}^\top \mathbf{x}\|^2]$$

Where we are?

SVM分类器



分类器
Classifier

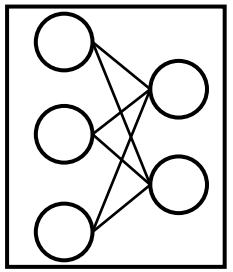


这里有无数个解平面！

哪一个最好 ***Best***？

Where we are?

SVM分类器



分类器
Classifier

SVM Objective

$$\min_{\boldsymbol{w},b} L(\boldsymbol{w},b) = \frac{1}{2}\|\boldsymbol{w}\|_2 + C\sum_i [1 - y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b)]_+$$

$$\nabla_{\boldsymbol{w}} L_i = \begin{cases} \boldsymbol{w} - Cy_i\boldsymbol{x}_i, & \text{若} 1 - y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) > 0 \\ \boldsymbol{w}, & \text{否则} \end{cases}$$

$$\nabla_b L_i = \begin{cases} -Cy_i, & \text{若} 1 - y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) > 0 \\ 0, & \text{否则} \end{cases}$$

基于梯度（Gradient）优化目标函数

大小为 N 的数据集$\{(\boldsymbol{x}_i, y_i)\}$，
随机初始 $\boldsymbol{w}$，b，总迭代次数 E

for e in range(E):

  for i in range(N):

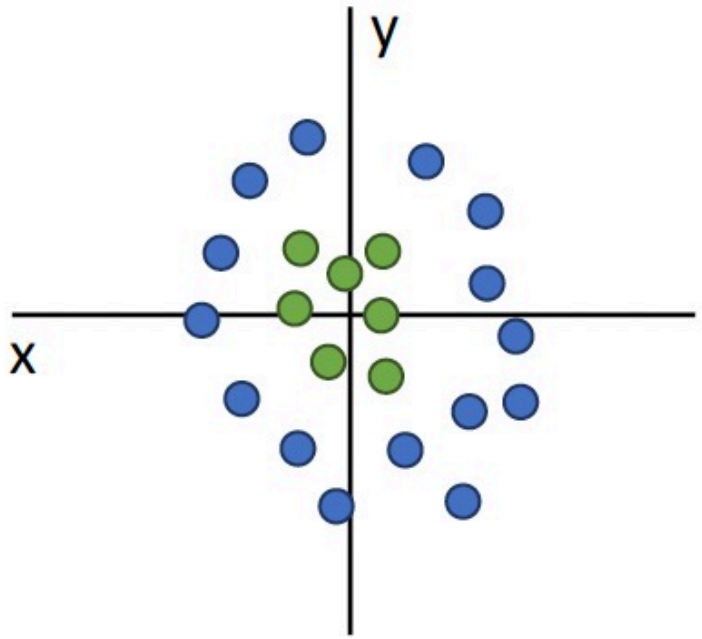
    if $1 - y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) > 0$:

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\nabla_{\boldsymbol{w}} L_i$$
$$b \leftarrow b - \eta\nabla_{\boldsymbol{b}} L_i$$

return $\boldsymbol{w}, b$

One Solution: **Feature Transforms**

Geometric Viewpoint



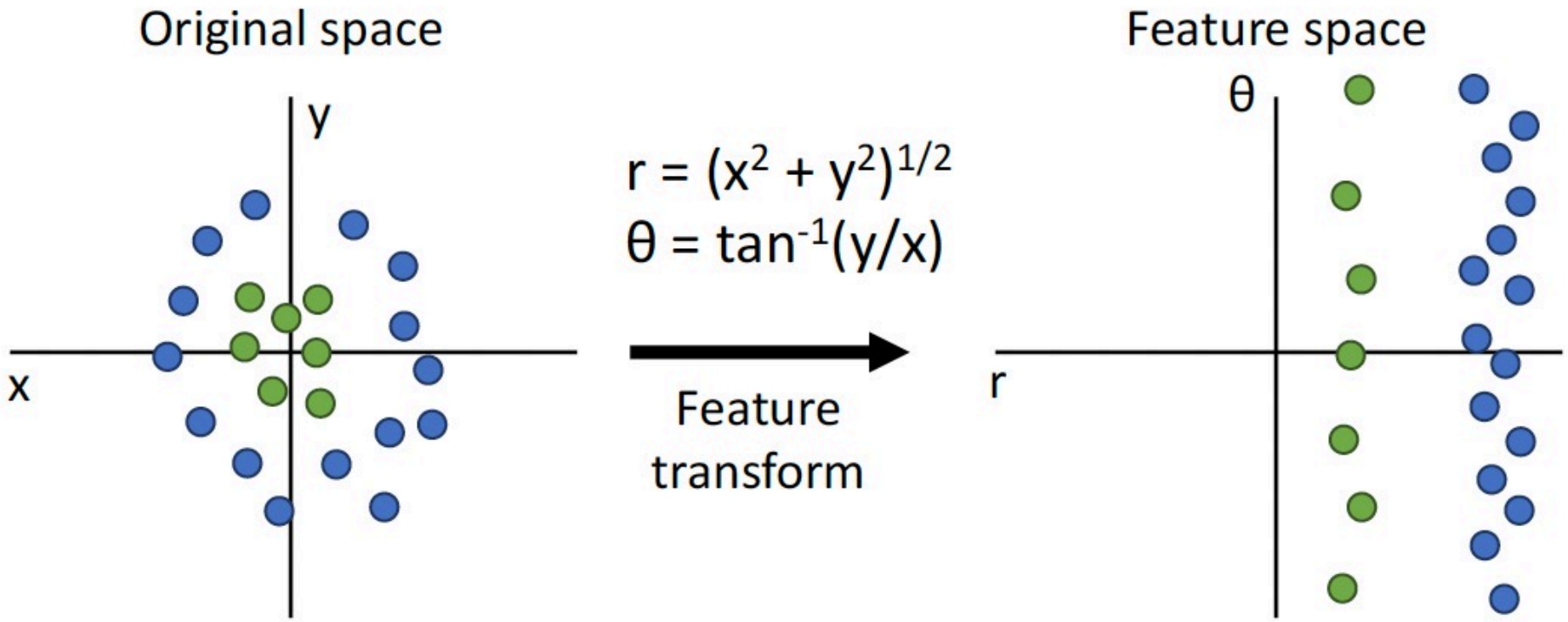
Visual Viewpoint

One template per class:
Can't recognize different
modes of a class

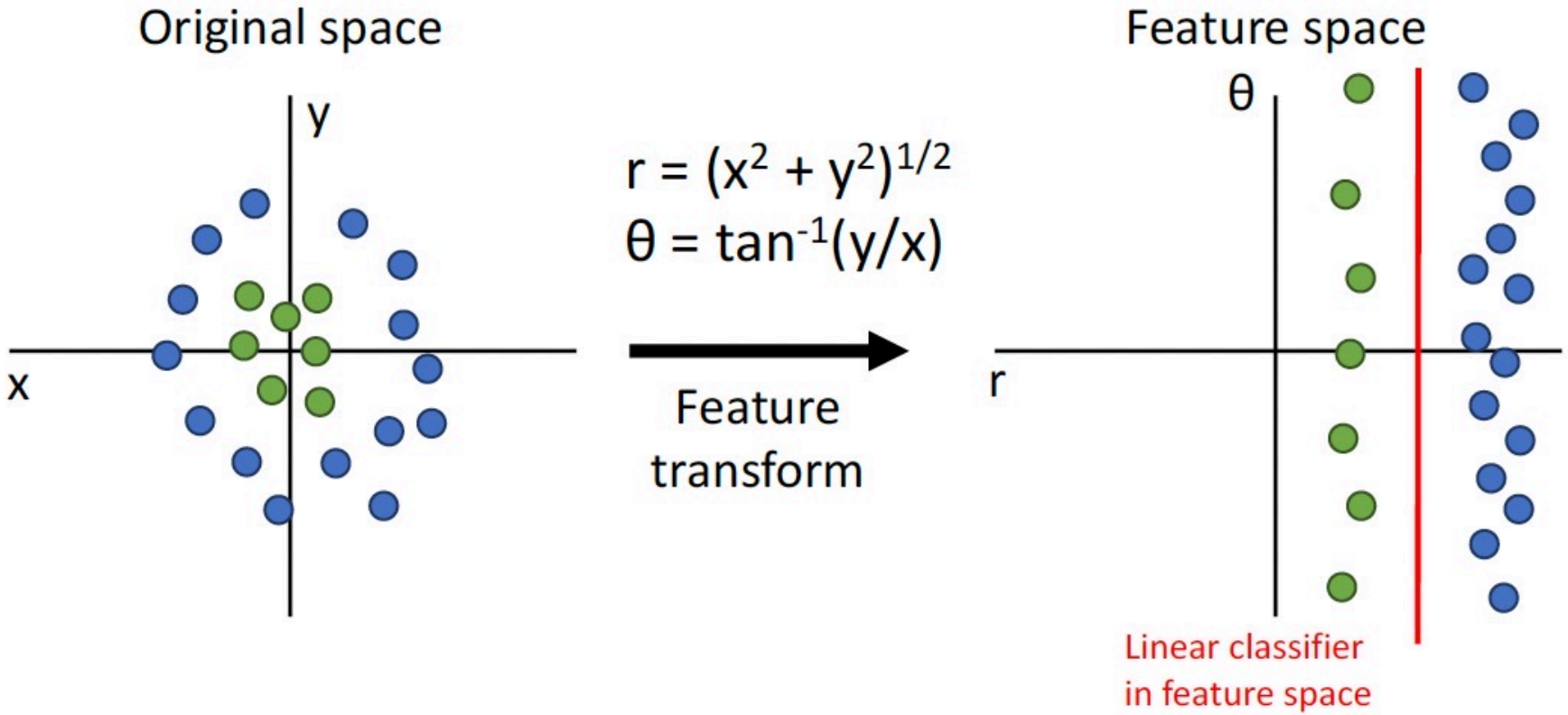


参考文献：https://web.eecs.umich.edu/~justincj/slides/eecs498/498_FA2019_lecture05.pdf

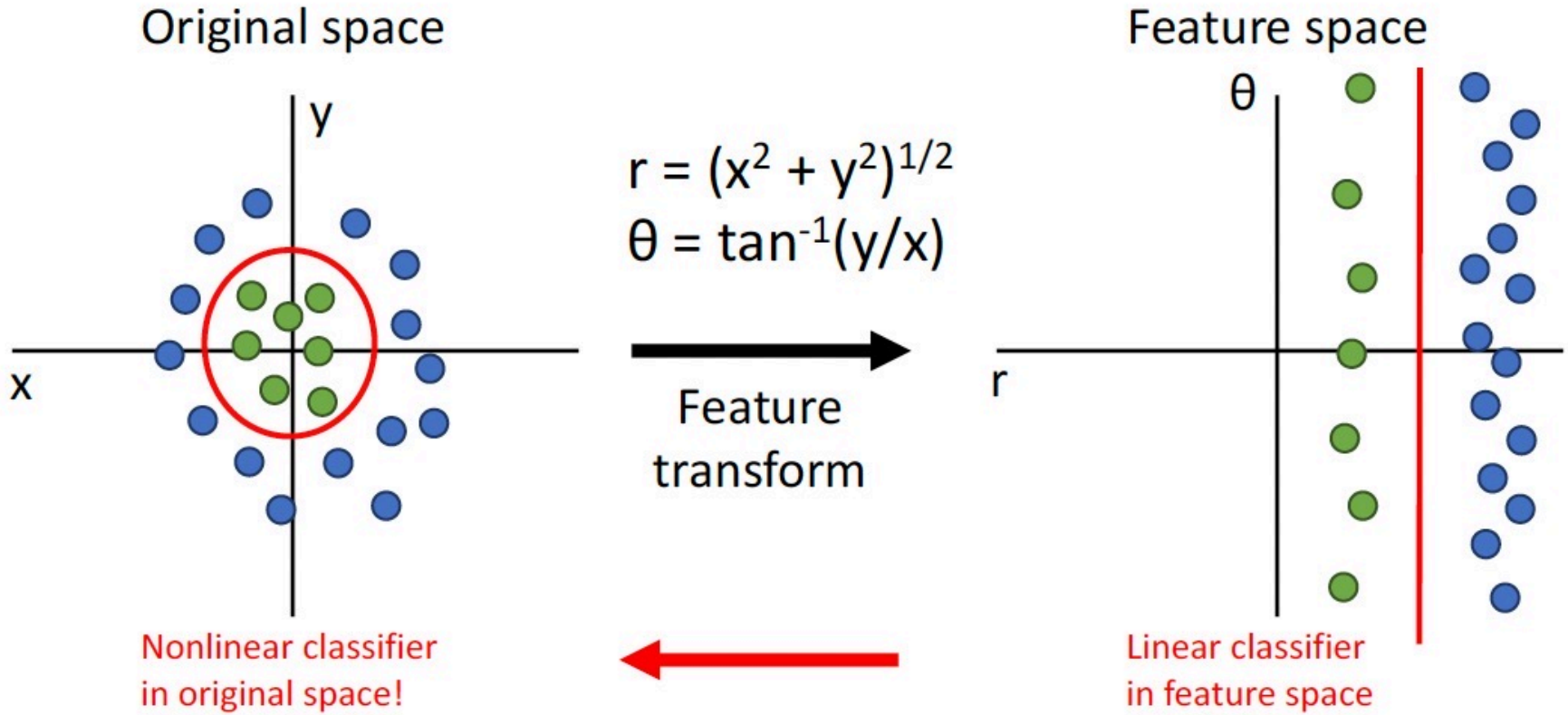
One Solution: **Feature Transforms**



One Solution: **Feature Transforms**

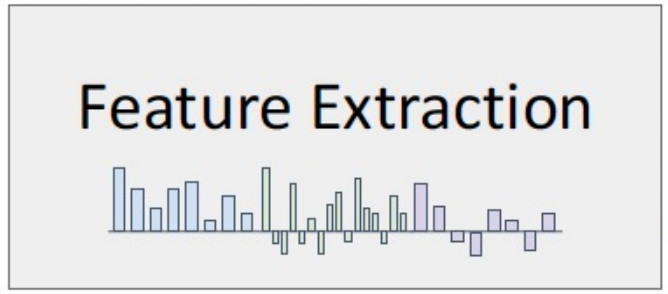


One Solution: **Feature Transforms**



Neural Networks



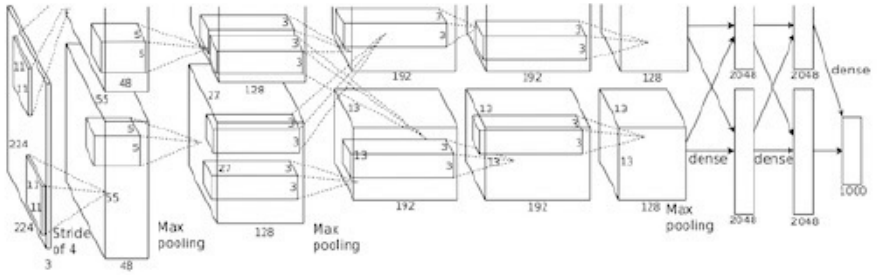


f

10 numbers giving scores for classes

training





Krizhevsky, Sutskever, and Hinton, "Imagenet classification with deep convolutional neural networks", NIPS 2012. Figure copyright Krizhevsky, Sutskever, and Hinton, 2012. Reproduced with permission.

10 numbers giving scores for classes

training

Neural Networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

实践中，通常会有可学习偏置（bias）b

Neural Networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

or 3-layer Neural Network

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

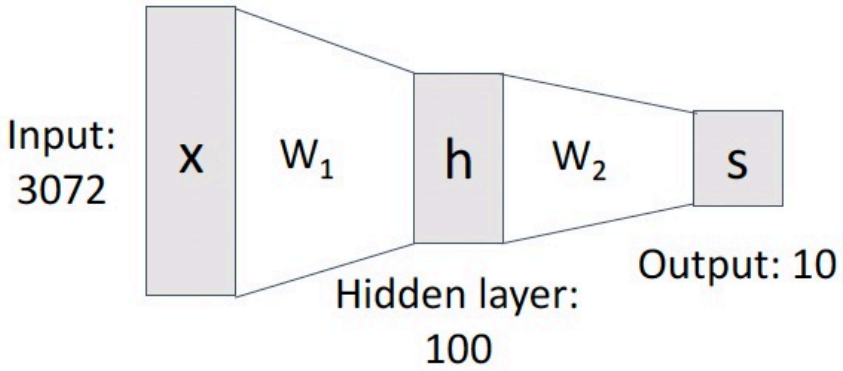
$W_3 \in \mathbb{R}^{C \times H_2}$ $W_2 \in \mathbb{R}^{H_2 \times H_1}$ $W_1 \in \mathbb{R}^{H_1 \times D}$ $x \in \mathbb{R}^D$

实践中，通常会有可学习偏置（bias）b

Neural Networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



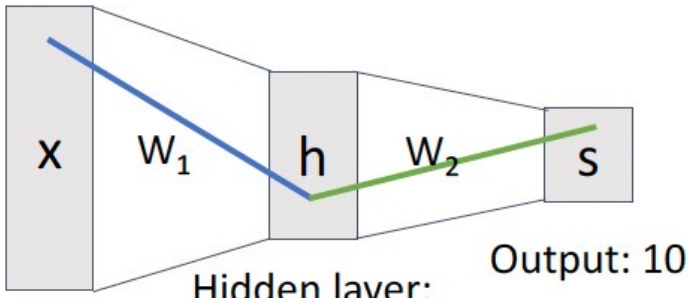
$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$

Neural Networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

Element (i, j) of W_1 gives the effect on h_i from x_j



Element (i, j) of W_2 gives the effect on s_i from h_j

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

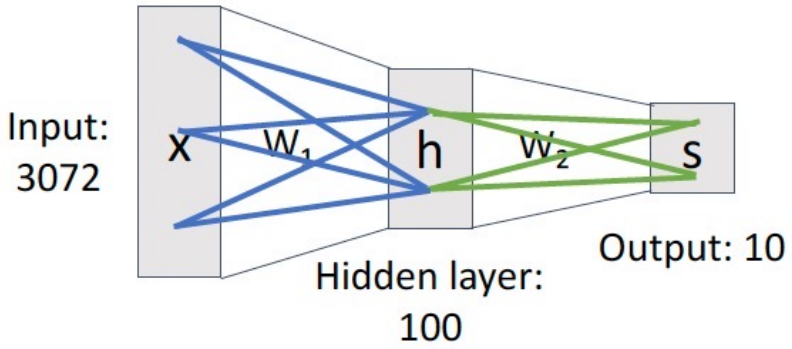
Neural Networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

Element (i, j) of W_1 gives the effect on h_i from x_j

All elements of x affect all elements of h



Element (i, j) of W_2 gives the effect on s_i from h_j

All elements of h affect all elements of s

Fully-connected neural network

Also “Multi-Layer Perceptron” (MLP)

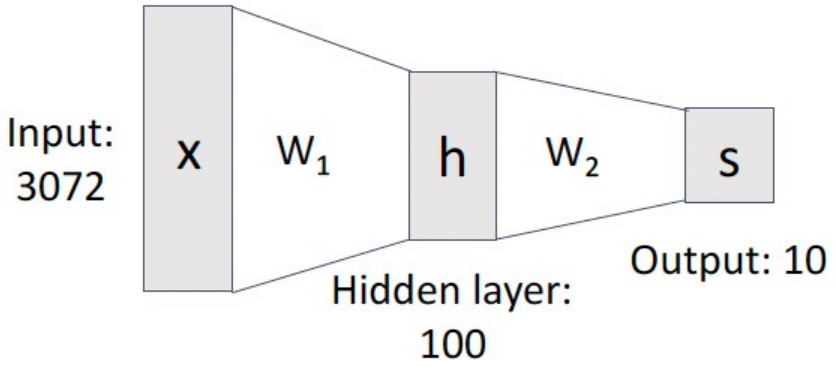
Neural Networks

Linear classifier: One template per class



(**Before**) Linear score function:

(**Now**) 2-layer Neural Network



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

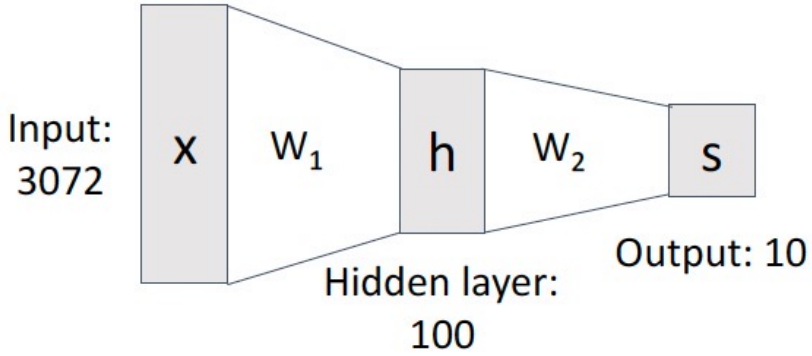
Neural Networks

Neural net: first layer is bank of templates;
Second layer recombines templates



(**Before**) Linear score function:

(**Now**) 2-layer Neural Network



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

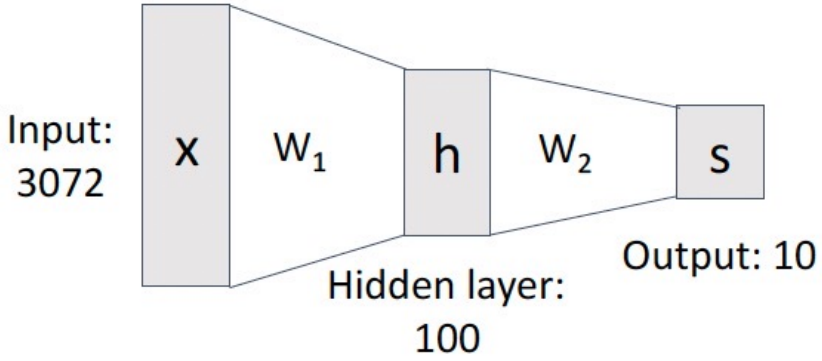
Neural Networks

Can use different templates to cover multiple modes of a class!



(**Before**) Linear score function:

(**Now**) 2-layer Neural Network



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

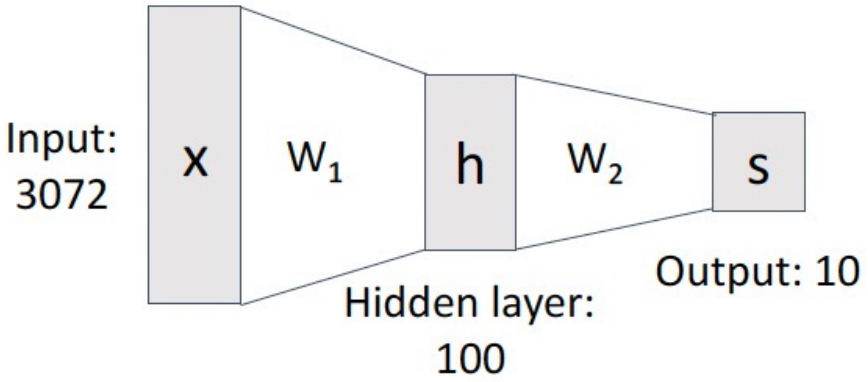
Neural Networks

"Distributed representation":
Most templates not interpretable!



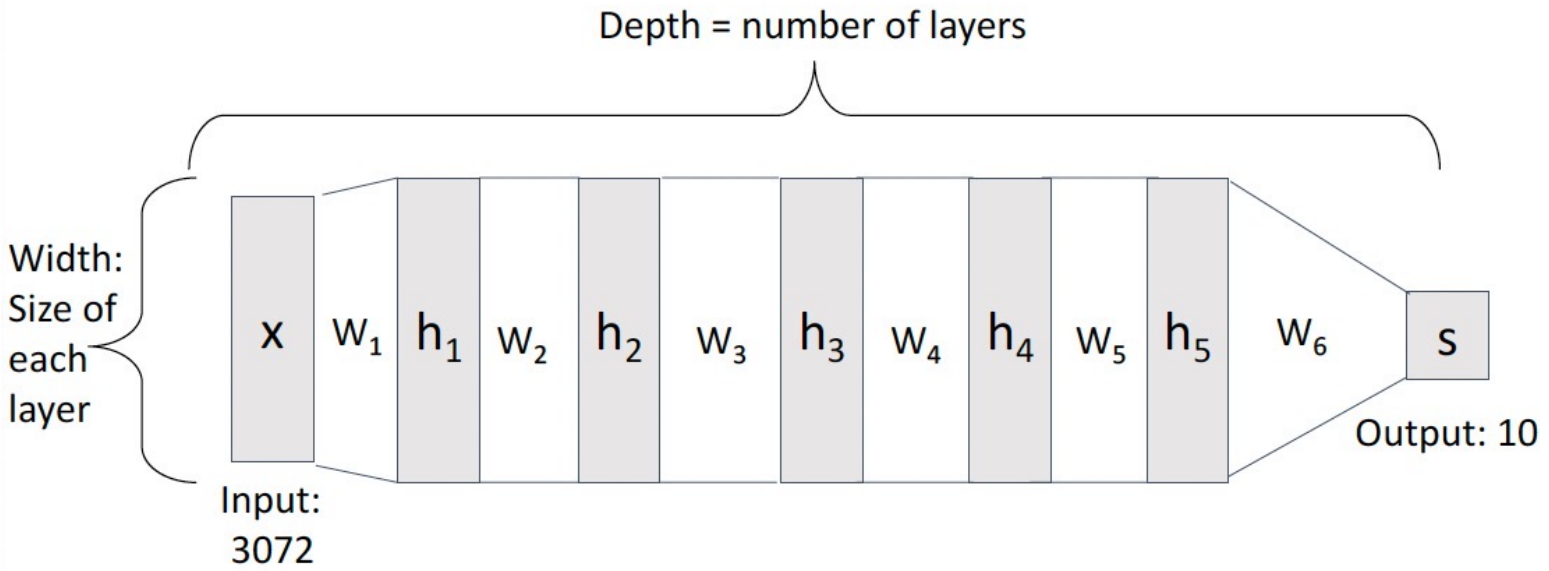
(**Before**) Linear score function:

(**Now**) 2-layer Neural Network



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Deep Neural Networks

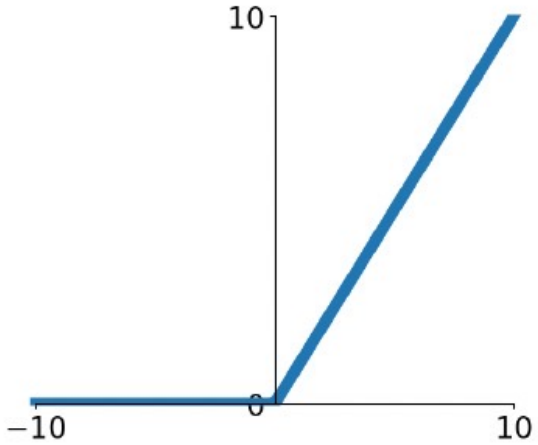


$$s = W_6 \max(0, W_6 \max(0, W_5 \max(0, W_4 \max(0, W_3 \max(0, W_2 \max(0, W_1 x))))))$$

Activation Functions

2-layer Neural Network

The function $ReLU(z) = max(0, z)$ is called "Rectified Linear Unit"



$$f = W_2 \max(0, W_1 x)$$

This is called the **activation function** of the neural network

Q: What happens if we build a neural network with no activation function?

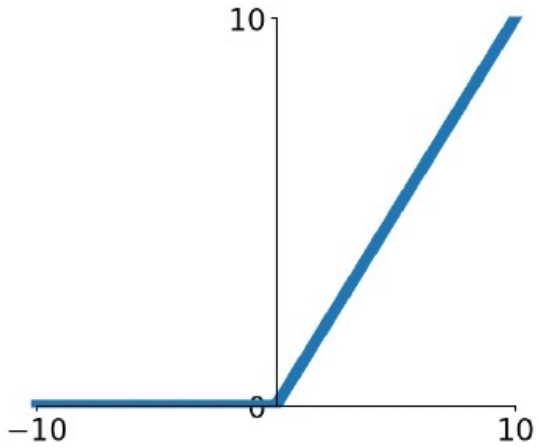
$$s = W_2 W_1 x$$

Activation Functions

2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

The function $ReLU(z) = max(0, z)$ is called "Rectified Linear Unit"



This is called the **activation function** of the neural network

Q: What happens if we build a neural network with no activation function?

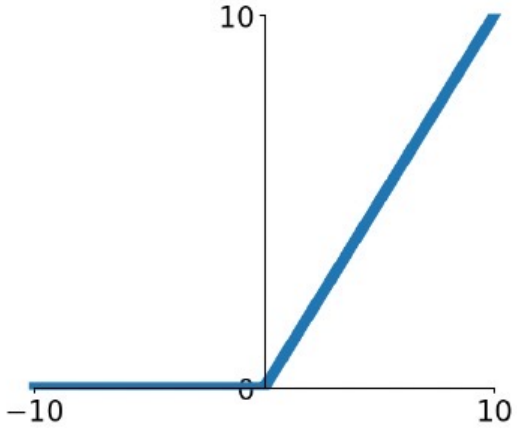
$$s = W_2 W_1 x$$

Activation Functions

2-layer Neural Network

$$f = W_2 \max(0, W_1 x)$$

The function $ReLU(z) = max(0, z)$ is called "Rectified Linear Unit"



This is called the **activation function** of the neural network

Q: What happens if we build a neural network with no activation function?

$$s = W_2 W_1 x$$

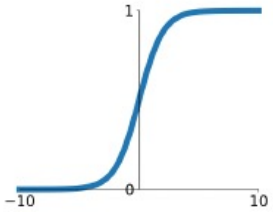
$$W_3 = W_2 W_1 \in \mathbb{R}^{C \times H} \qquad s = W_3 x$$

A: We end up with a linear classifier!

Activation Functions

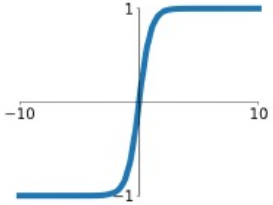
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



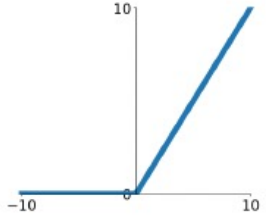
tanh

$$\tanh(x)$$



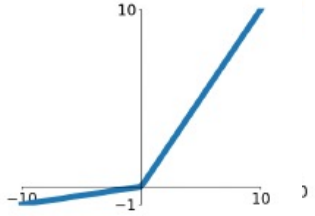
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

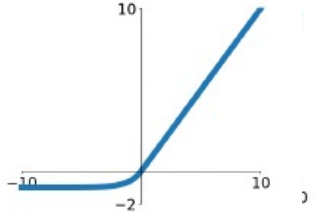


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

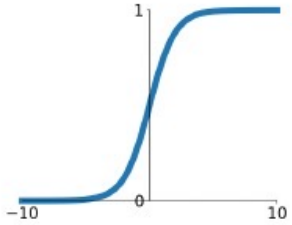


Activation Functions

ReLU is a good default choice for most problems

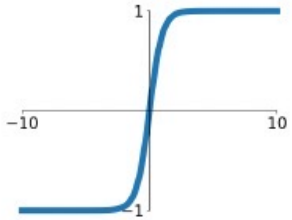
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



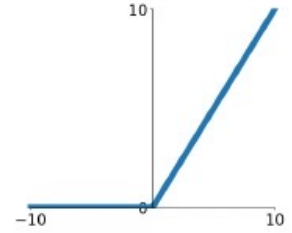
tanh

$$\tanh(x)$$



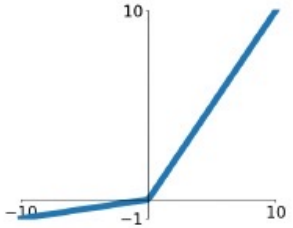
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

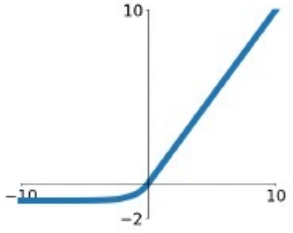


Maxout

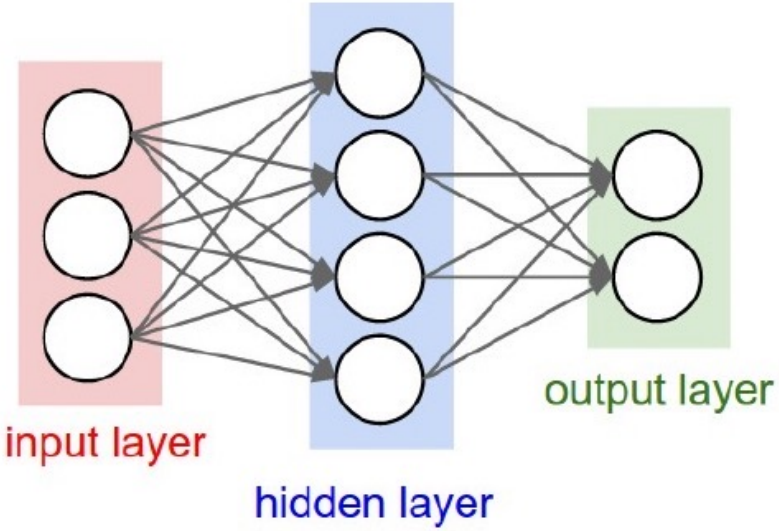
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



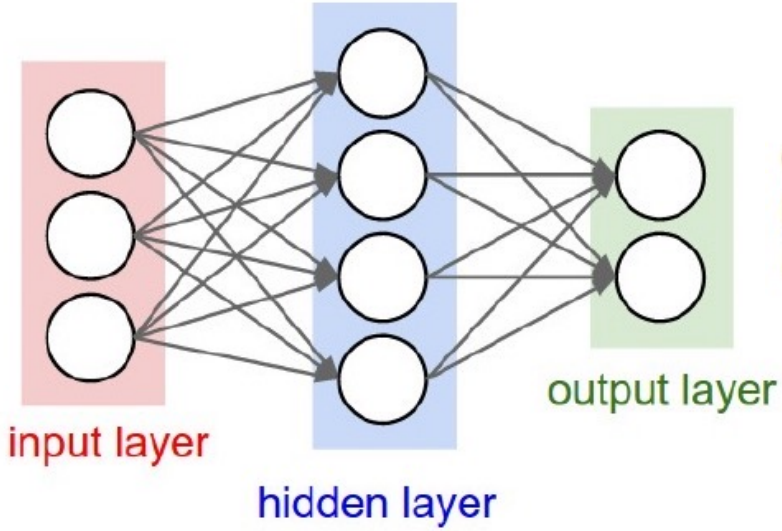
Neural Net in < 20 lines!




```python
import numpy as np
from numpy.random import randn

N, Din, H, Dout = 64, 1000, 100, 10
x, y = randn(N, Din), randn(N, Dout)
w1, w2 = randn(Din, H), randn(H, Dout)
for t in range(10000):
  h = 1.0 / (1.0 + np.exp(-x.dot(w1)))
  y_pred = h.dot(w2)
  loss = np.square(y_pred - y).sum()
  dy_pred = 2.0 * (y_pred - y)
  dw2 = h.T.dot(dy_pred)
  dh = dy_pred.dot(w2.T)
  dw1 = x.T.dot(dh * h * (1 - h))
  w1 -= 1e-4 * dw1
  w2 -= 1e-4 * dw2
```

Neural Net in < 20 lines!



Initialize weights and data

Compute loss (sigmoid activation, L2 loss)

Compute gradients

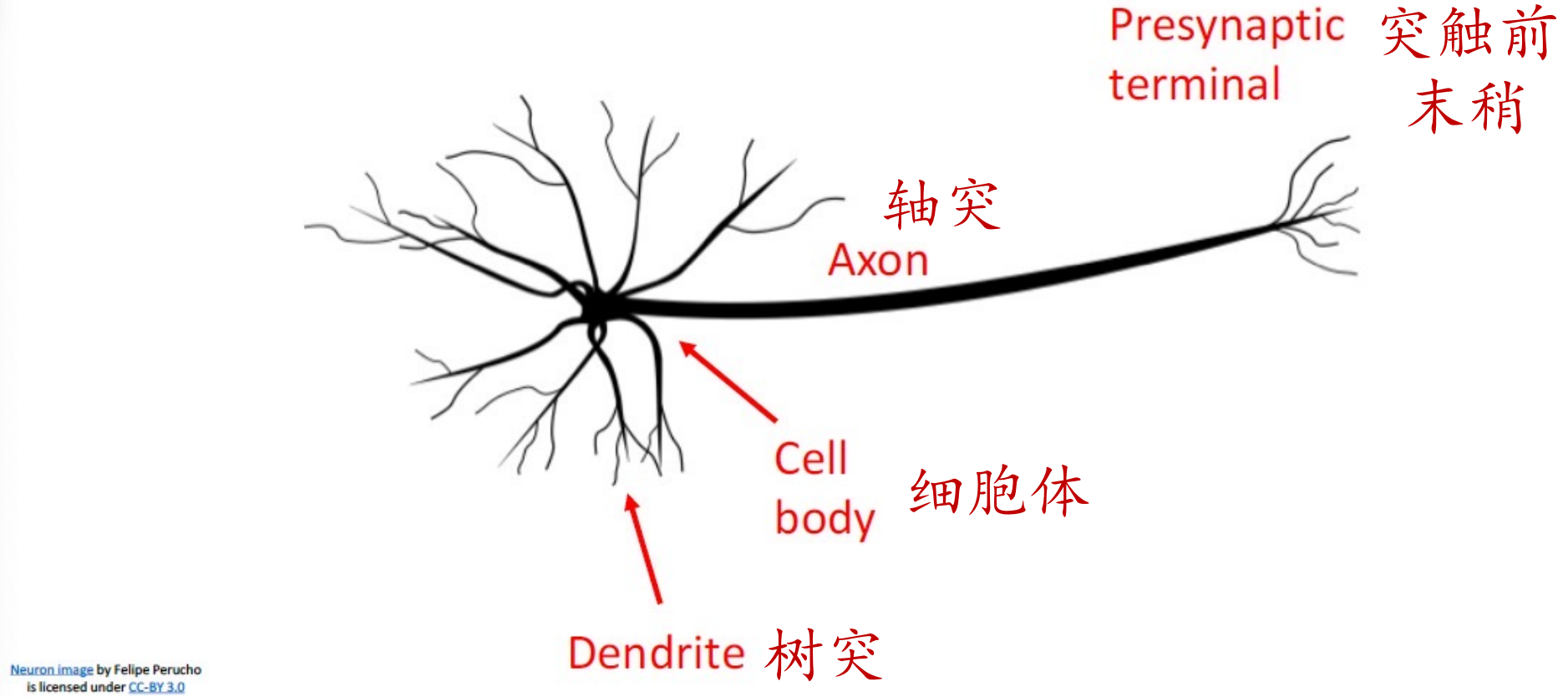
SGD step

```
1   import numpy as np
2   from numpy.random import randn
3
4   N, Din, H, Dout = 64, 1000, 100, 10
5   x, y = randn(N, Din), randn(N, Dout)
6   w1, w2 = randn(Din, H), randn(H, Dout)
7   for t in range(10000):
8     h = 1.0 / (1.0 + np.exp(-x.dot(w1)))
9     y_pred = h.dot(w2)
10    loss = np.square(y_pred - y).sum()
11    dy_pred = 2.0 * (y_pred - y)
12    dw2 = h.T.dot(dy_pred)
13    dh = dy_pred.dot(w2.T)
14    dw1 = x.T.dot(dh * h * (1 - h))
15    w1 -= 1e-4 * dw1
16    w2 -= 1e-4 * dw2
```



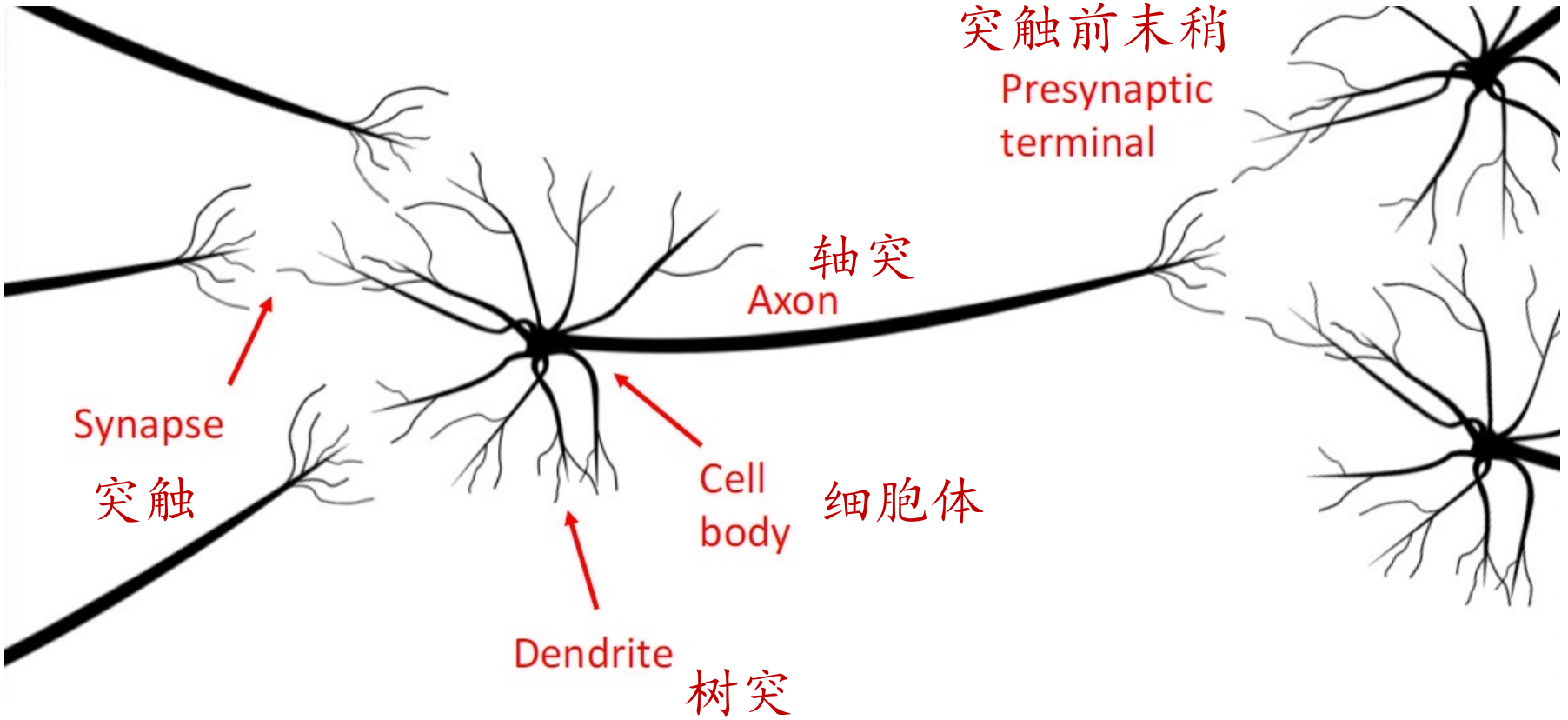
This image by Fotis Bobolas is licensed under CC-BY 2.0

Our brains are made of Neurons

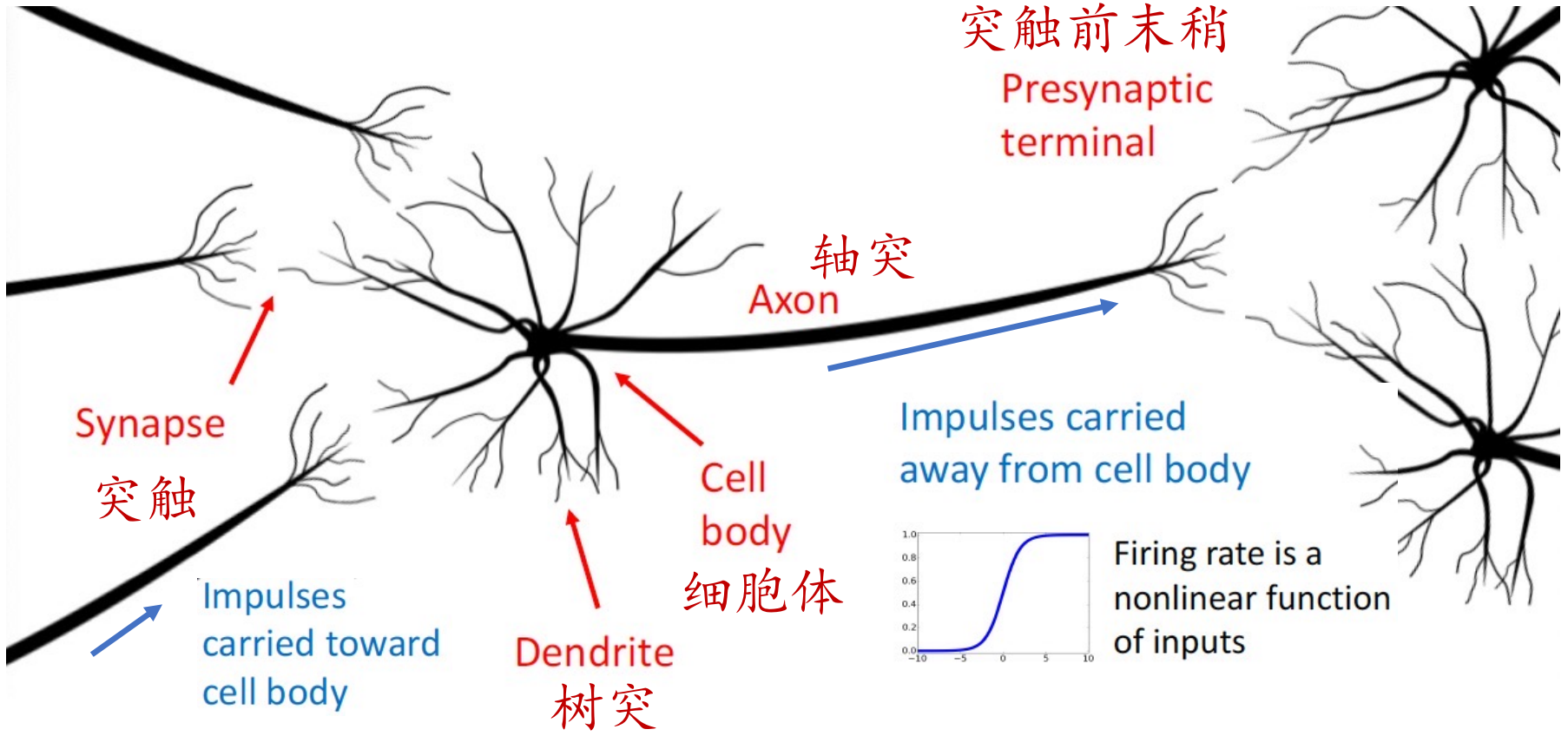


Neuron image by Felipe Perucho is licensed under CC-BY 3.0

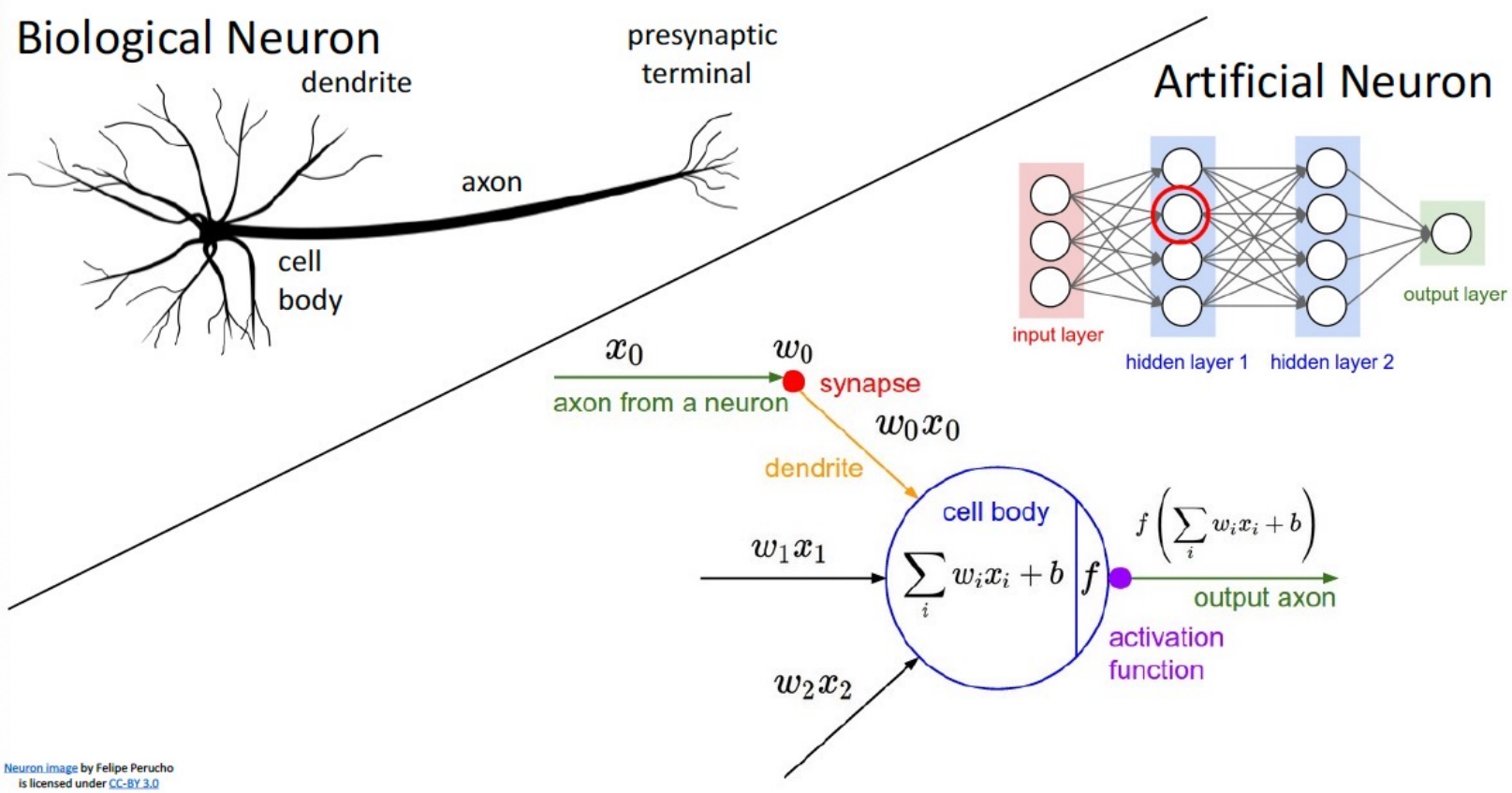
Our brains are made of Neurons



Our brains are made of Neurons

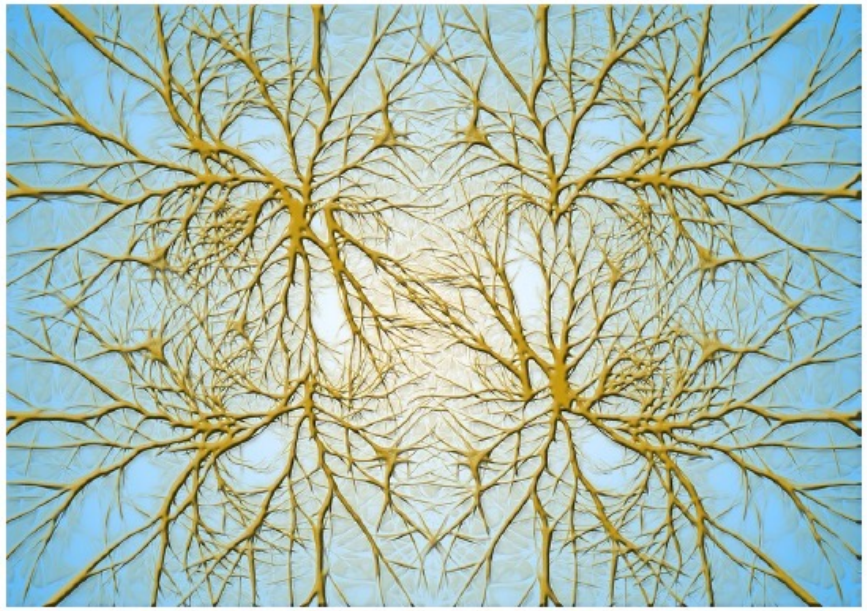


Our brains are made of Neurons



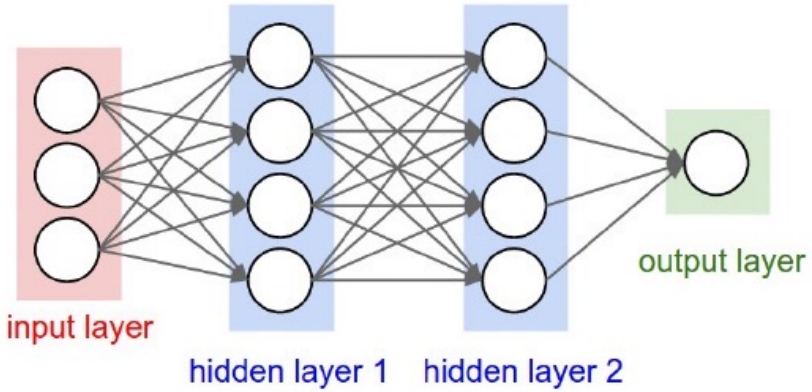
Our brains are made of Neurons

Biological Neurons:
Complex connectivity patterns



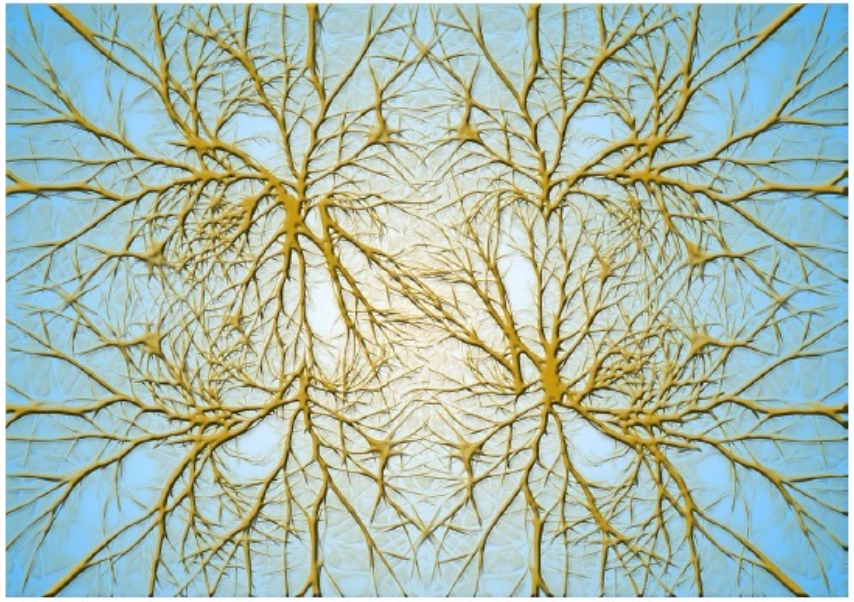
This image is CC0 Public Domain

Neurons in a neural network:
Organized into regular layers for computational efficiency



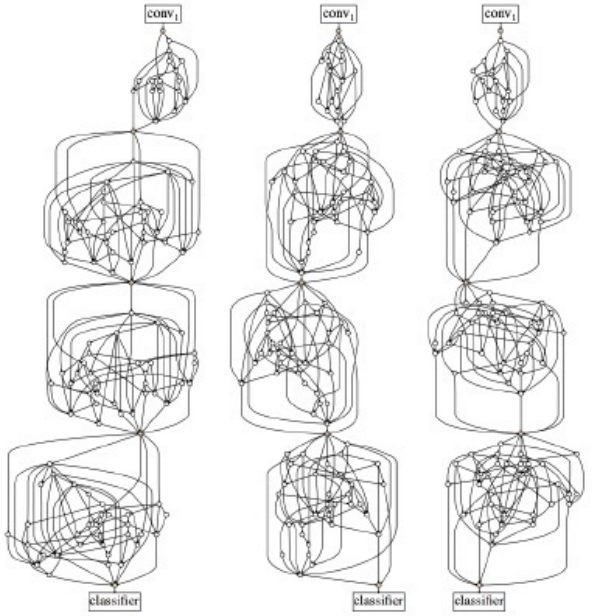
Our brains are made of Neurons

Biological Neurons:
Complex connectivity patterns



This image is CC0 Public Domain

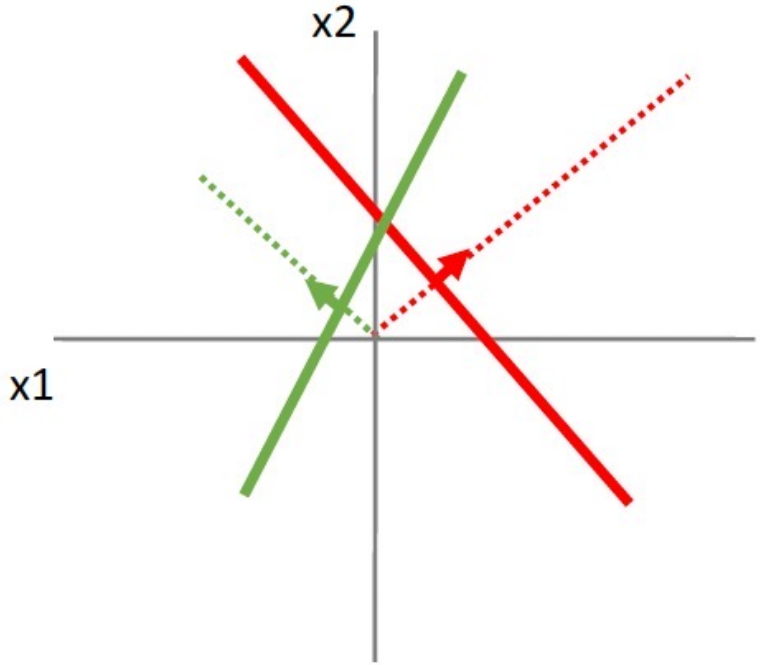
But neural networks with random connections can work too!



Xie et al, "Exploring Randomly Wired Neural Networks for Image Recognition", ICCV 2019

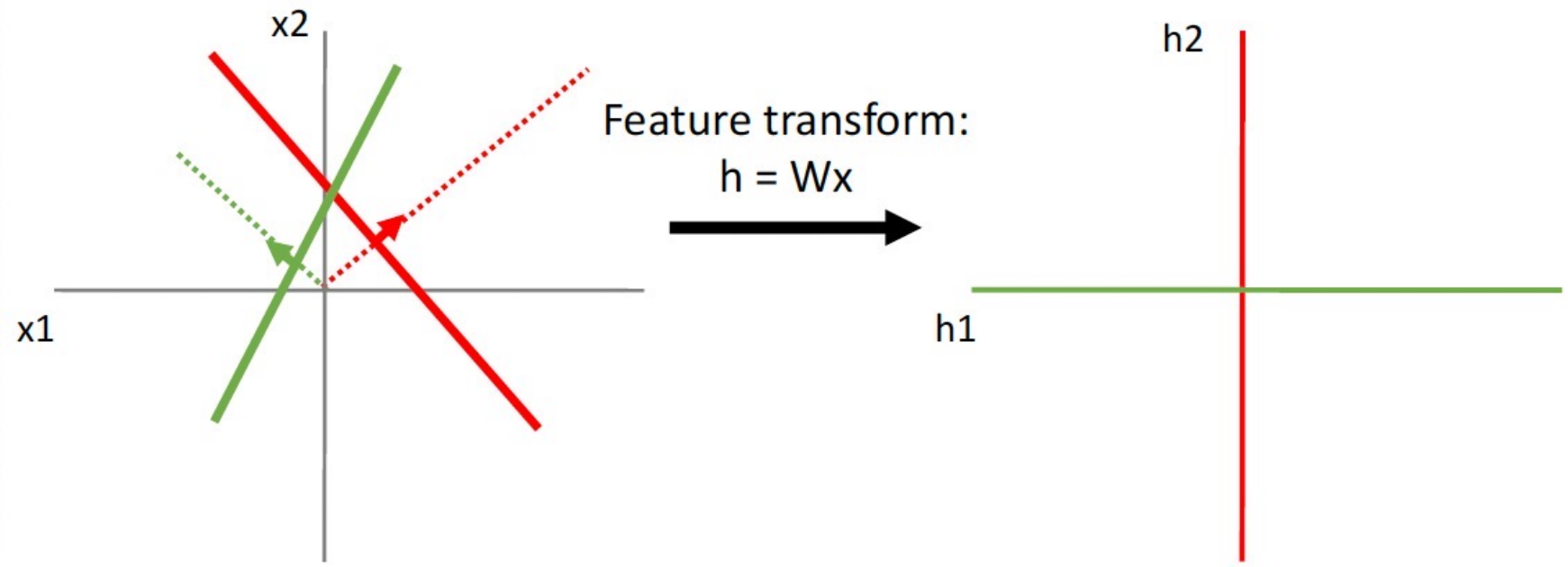
Space Warping

Consider a linear transform: h = Wx
Where x, h are both 2-dimensional



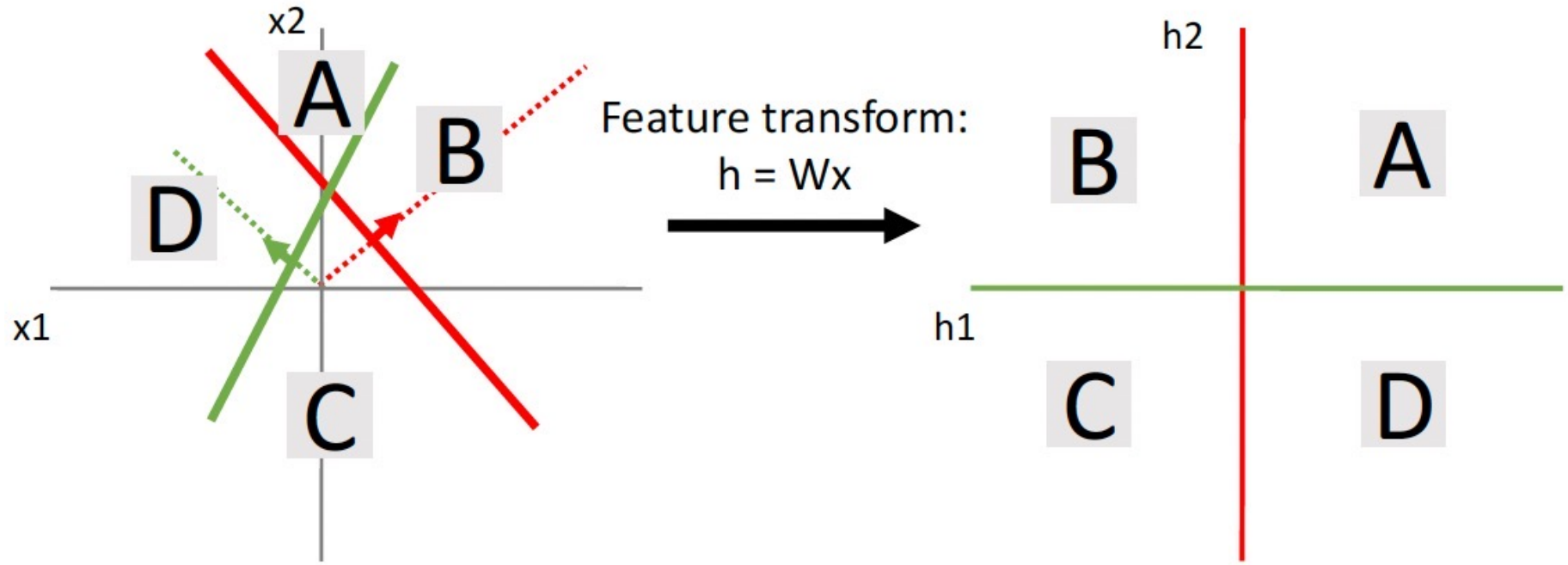
Space Warping

Consider a linear transform: h = Wx
Where x, h are both 2-dimensional

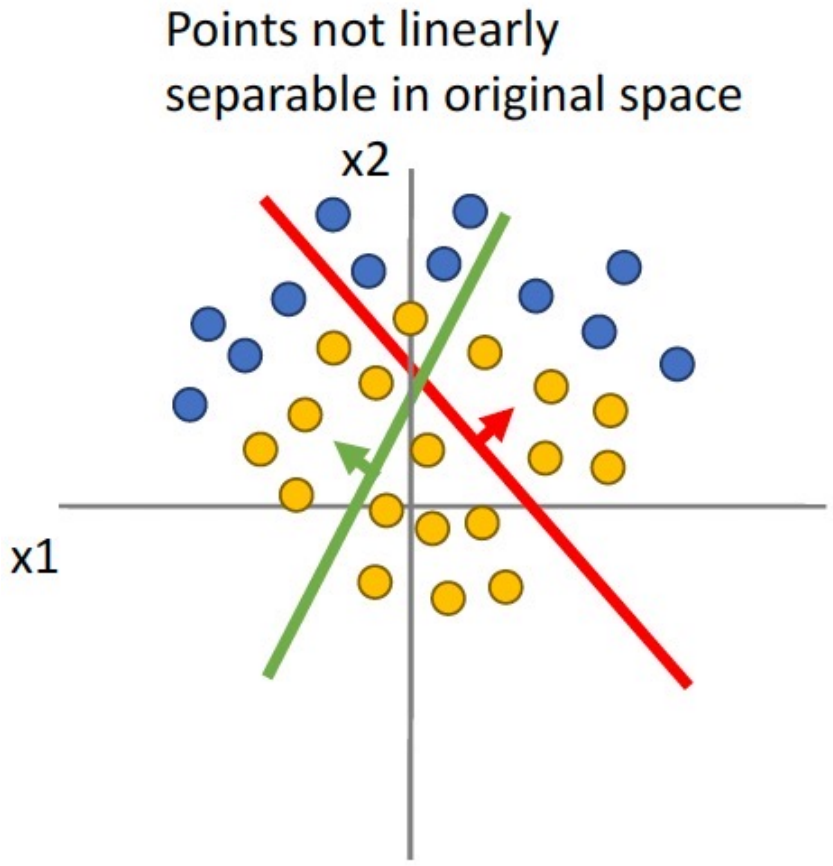


Space Warping

Consider a linear transform: h = Wx
Where x, h are both 2-dimensional

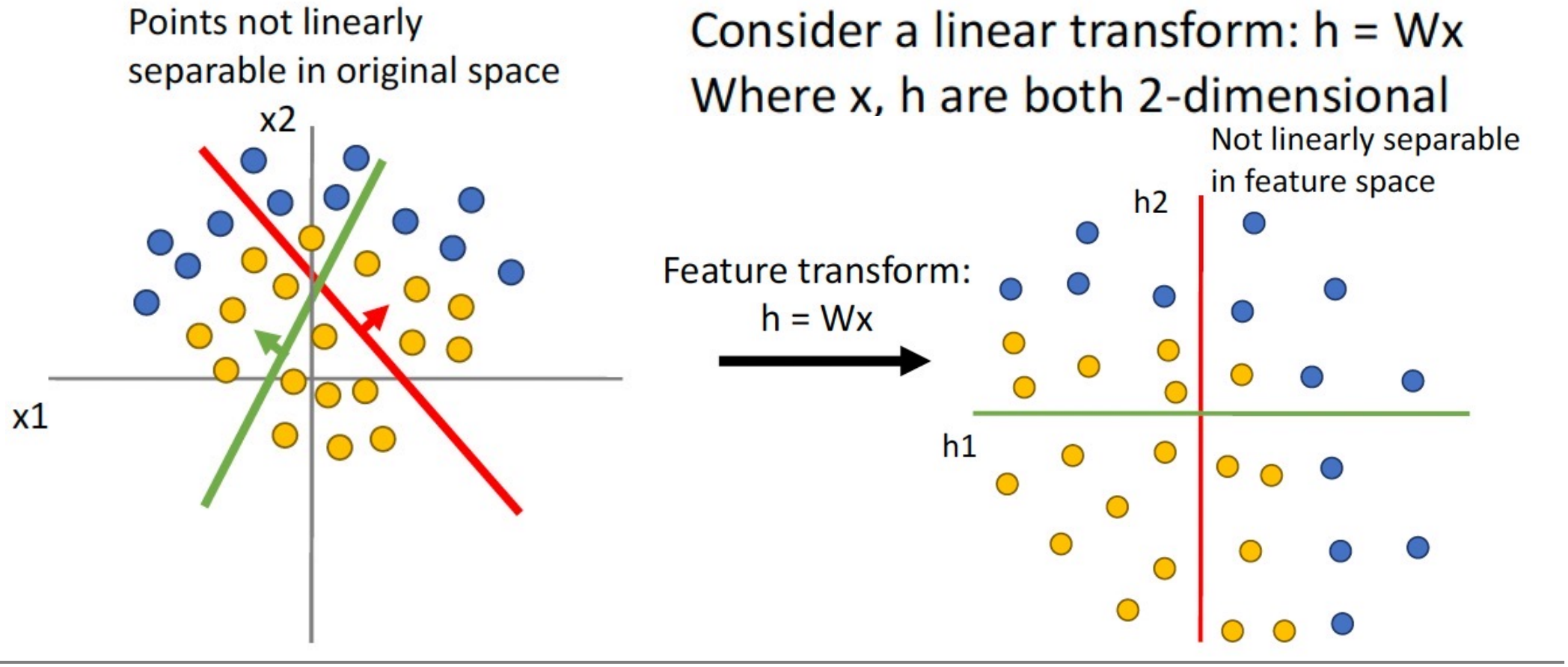


Space Warping



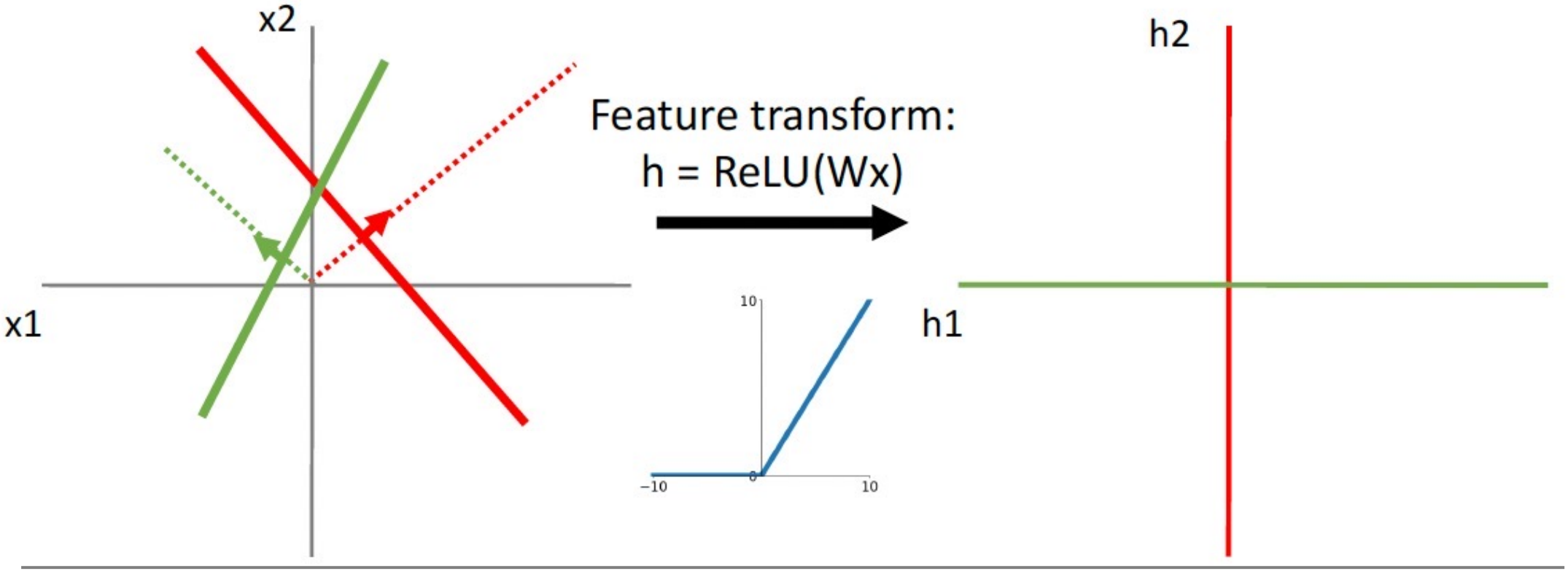
Consider a linear transform: h = Wx
Where x, h are both 2-dimensional

Space Warping



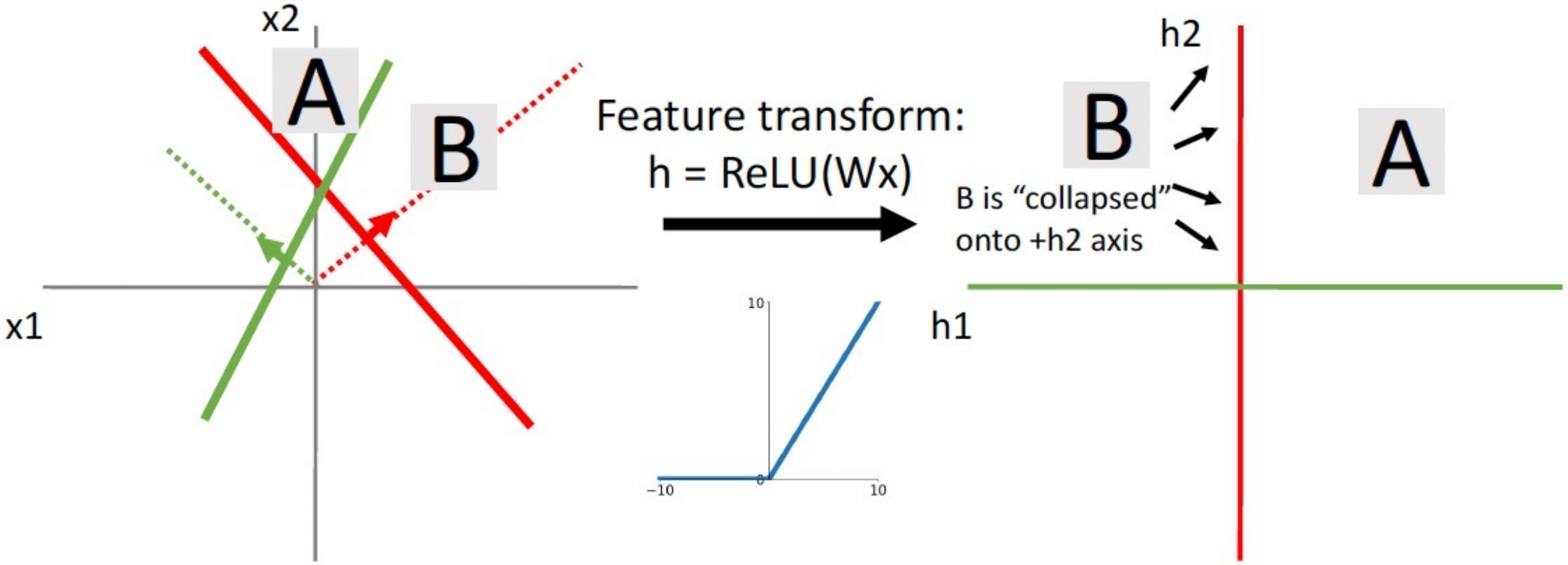
Space Warping

Consider a neural net hidden layer:
h = ReLU(Wx) = max(0, Wx)
Where x, h are both 2-dimensional



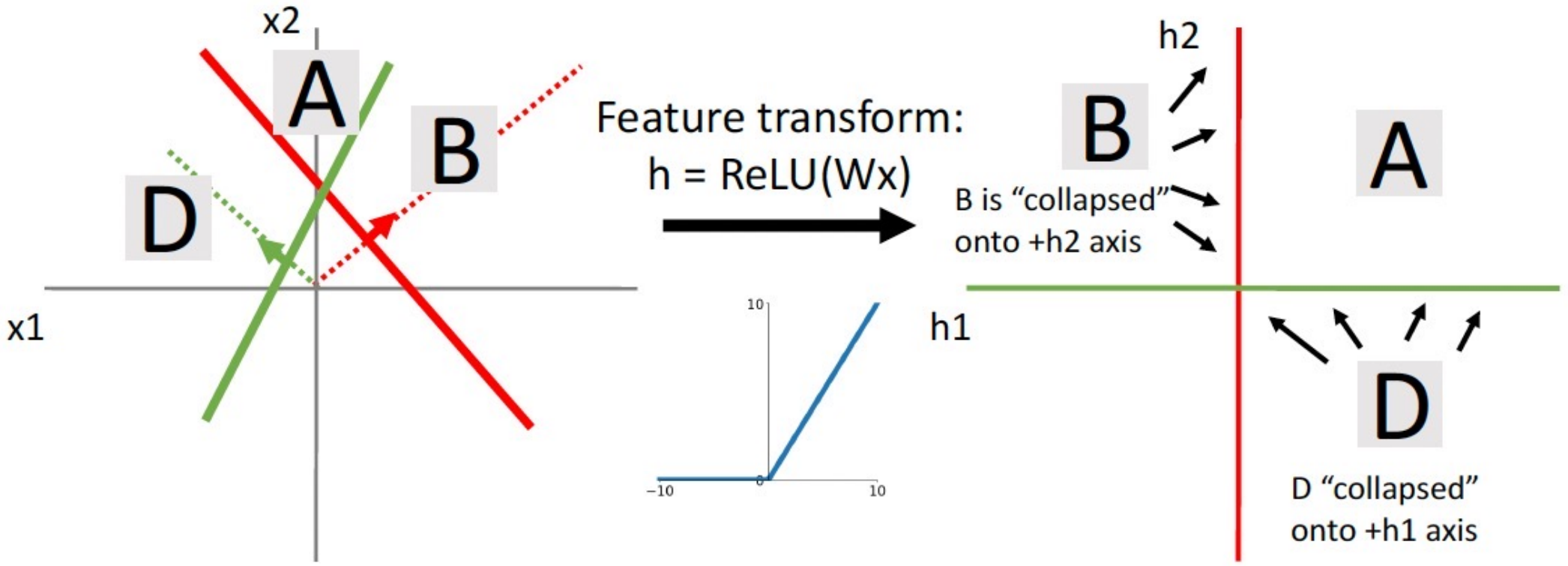
Space Warping

Consider a neural net hidden layer:
h = ReLU(Wx) = max(0, Wx)
Where x, h are both 2-dimensional



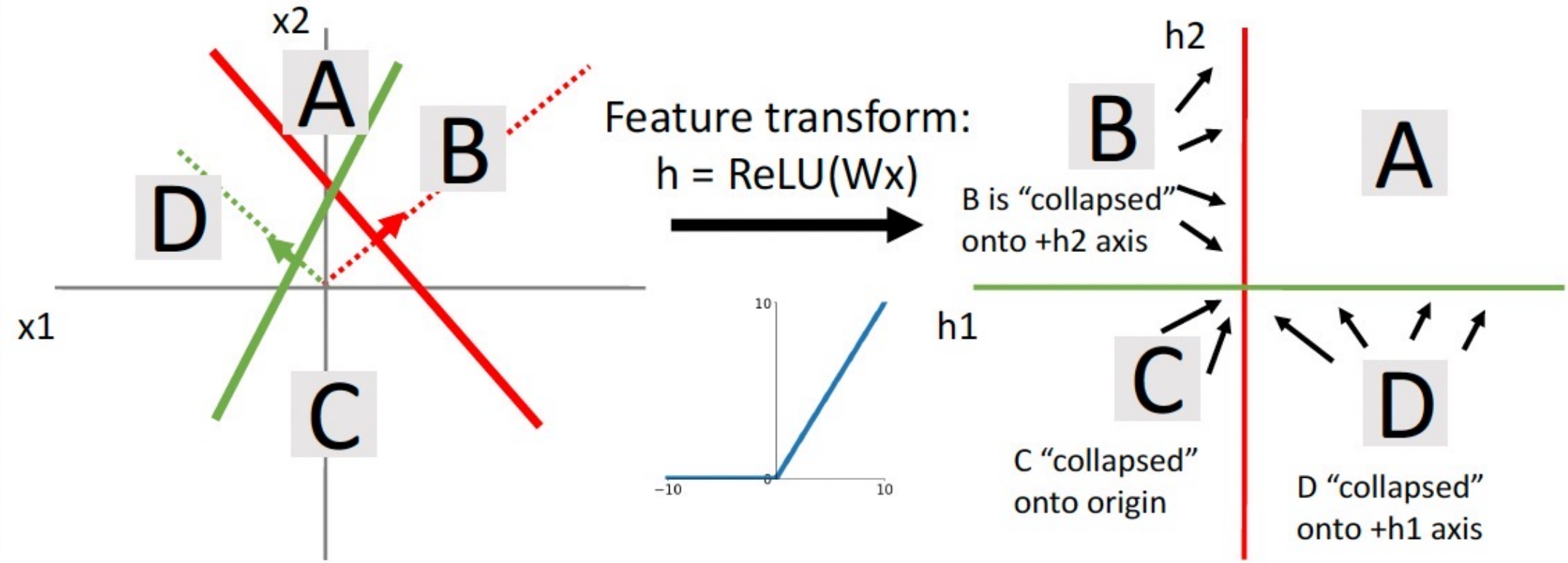
Space Warping

Consider a neural net hidden layer:
h = ReLU(Wx) = max(0, Wx)
Where x, h are both 2-dimensional



Space Warping

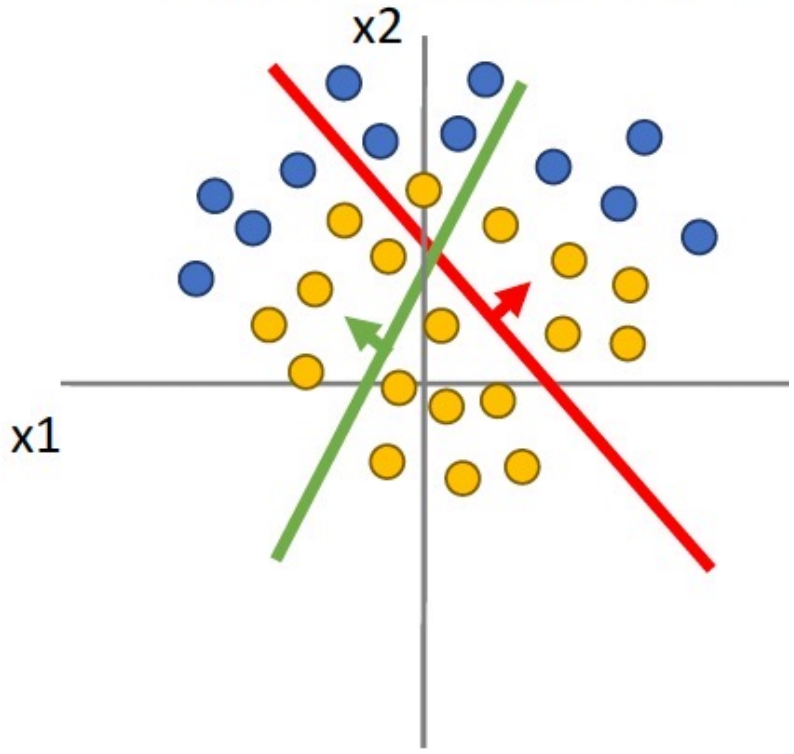
Consider a neural net hidden layer:
h = ReLU(Wx) = max(0, Wx)
Where x, h are both 2-dimensional



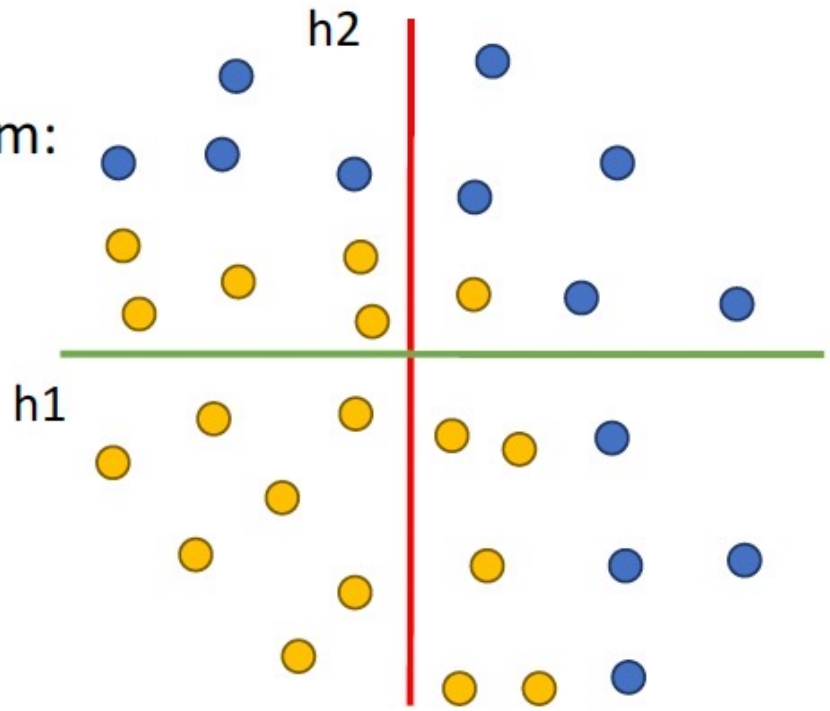
Space Warping

Consider a neural net hidden layer:
h = ReLU(Wx) = max(0, Wx)
Where x, h are both 2-dimensional

Points not linearly separable in original space



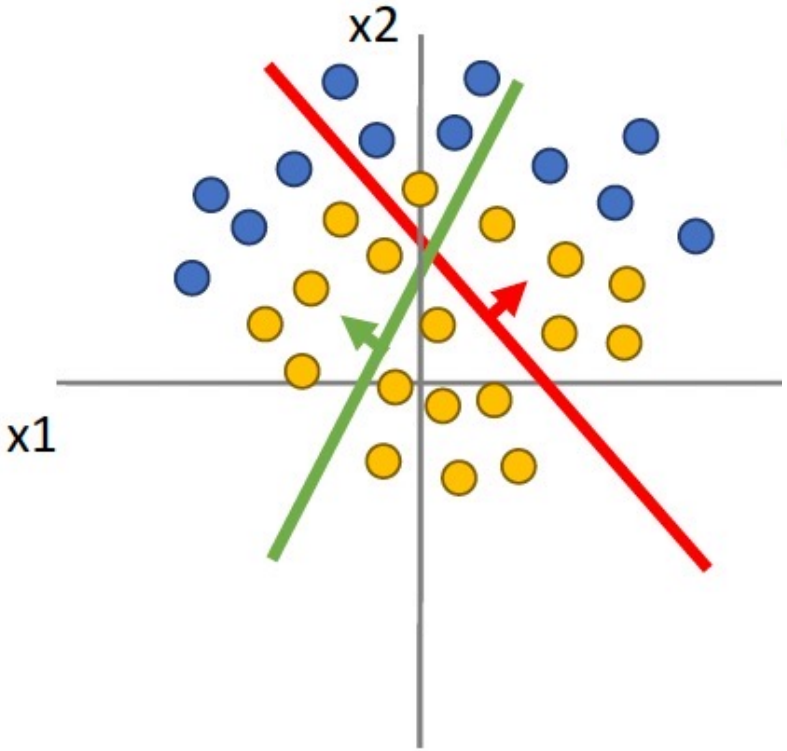
Feature transform:
h = Wx



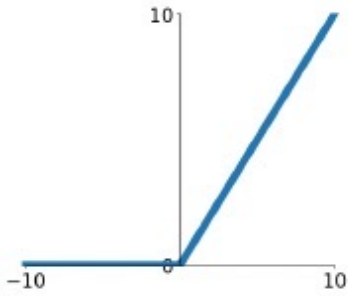
Space Warping

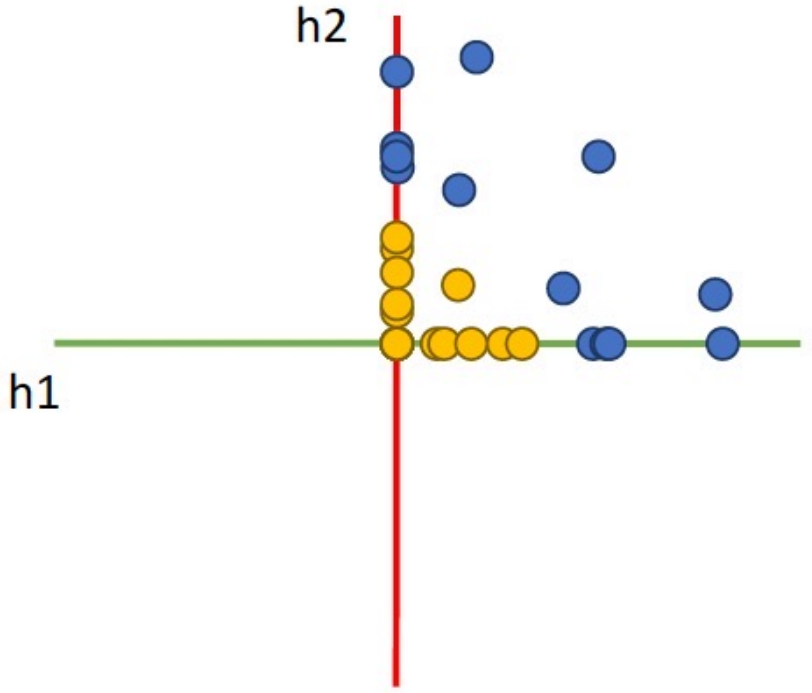
Consider a neural net hidden layer:
h = ReLU(Wx) = max(0, Wx)
Where x, h are both 2-dimensional

Points not linearly separable in original space



Feature transform:
h = ReLU(Wx)

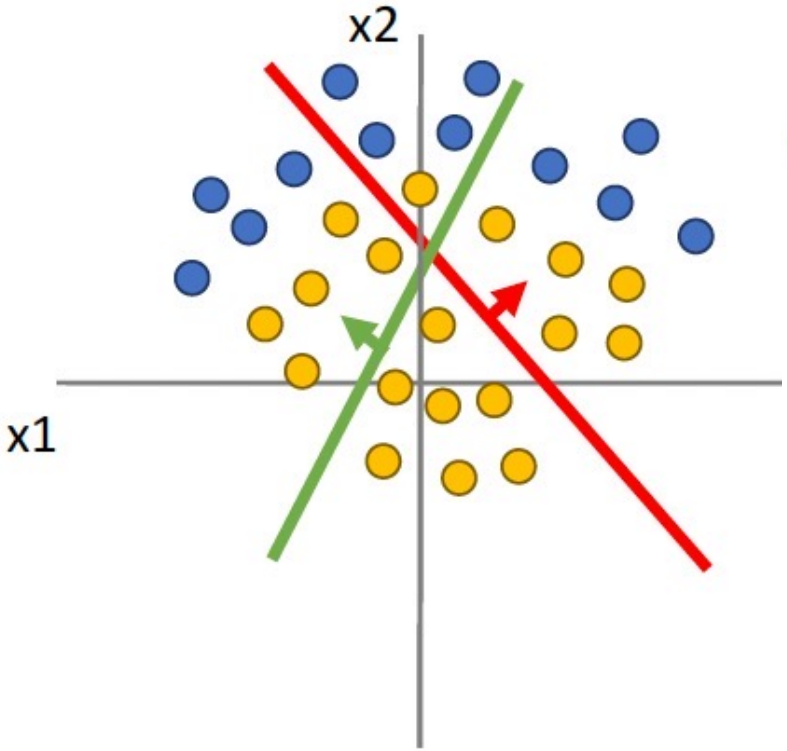




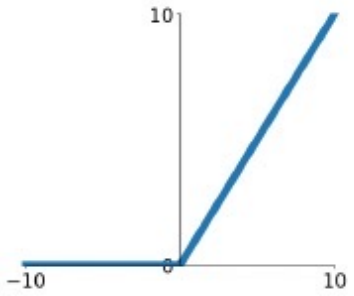
Space Warping

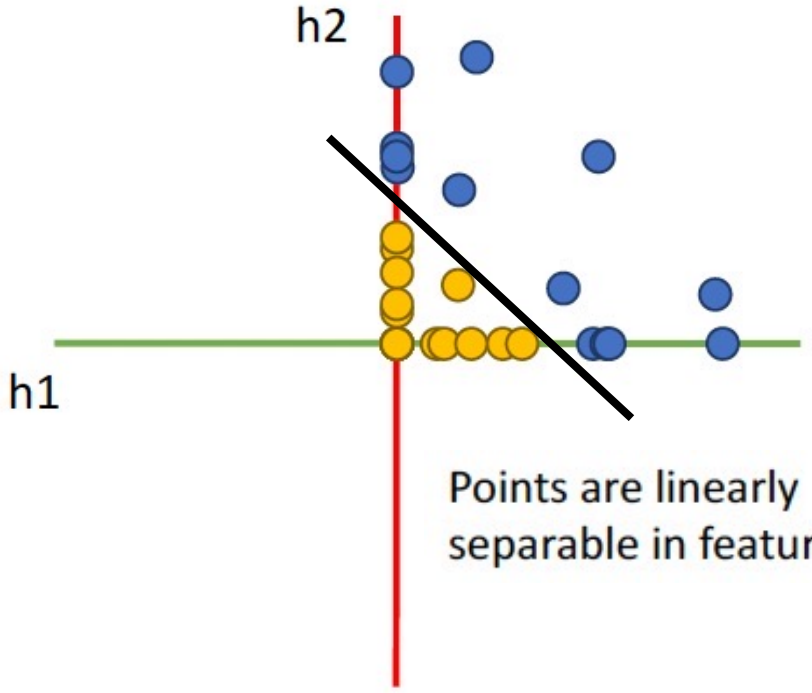
Consider a neural net hidden layer:
h = ReLU(Wx) = max(0, Wx)
Where x, h are both 2-dimensional

Points not linearly separable in original space



Feature transform:
h = ReLU(Wx)



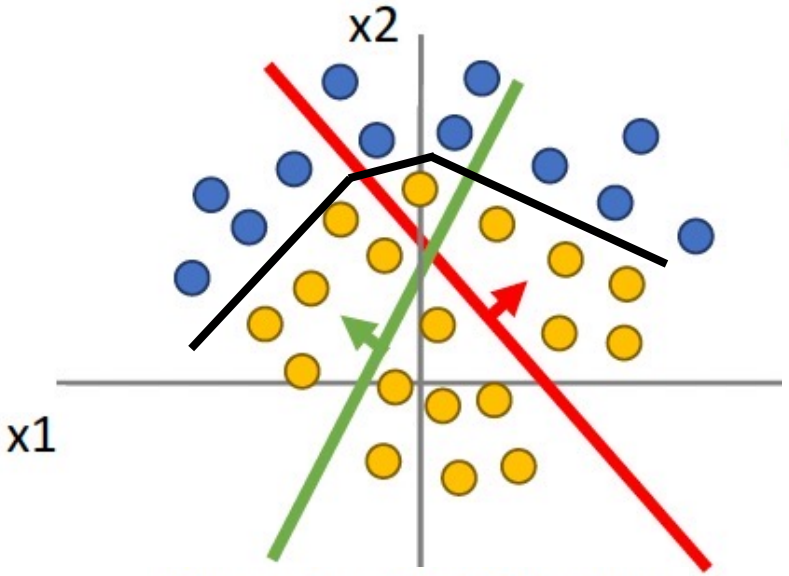


Points are linearly separable in features space!

Space Warping

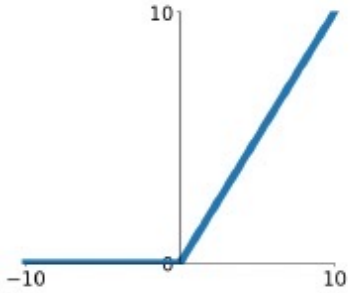
Consider a neural net hidden layer:
h = ReLU(Wx) = max(0, Wx)
Where x, h are both 2-dimensional

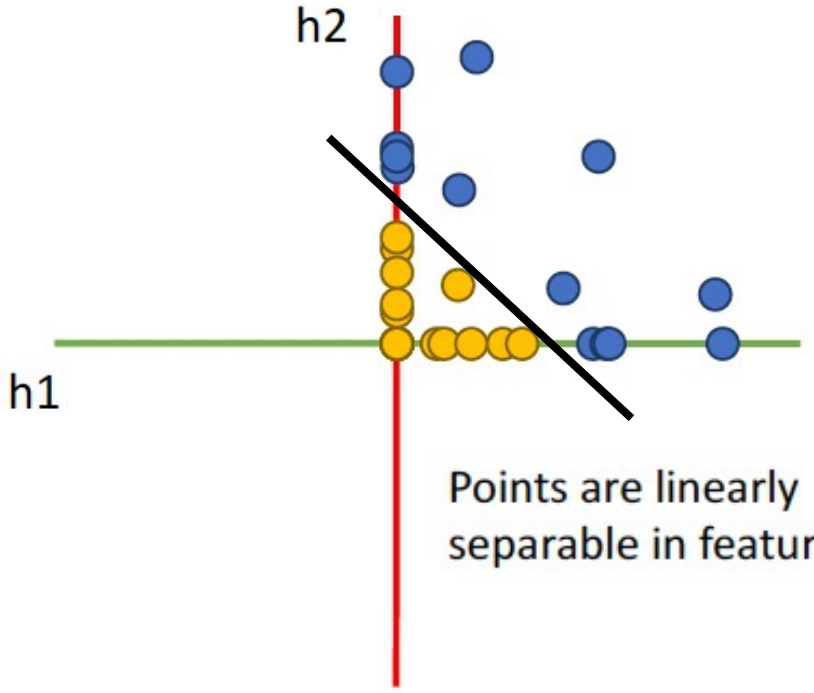
Points not linearly separable in original space



Linear classifier in feature space gives nonlinear classifier in original space

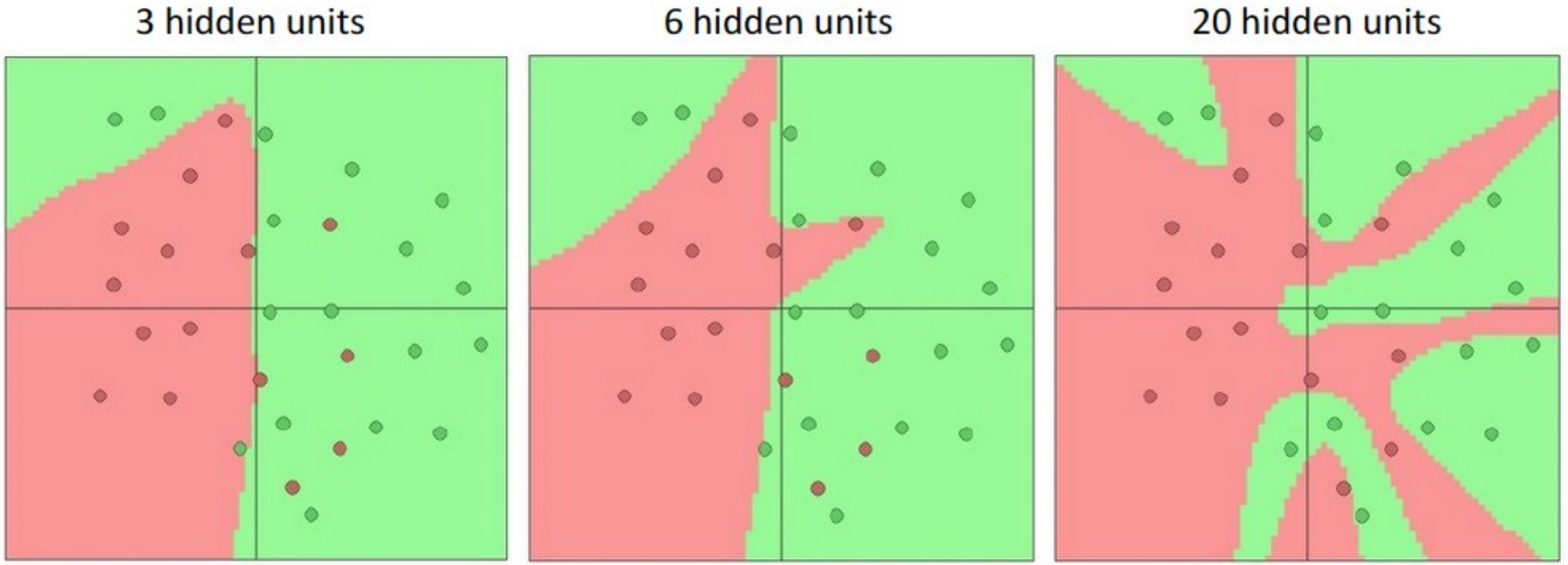
Feature transform:
h = ReLU(Wx)





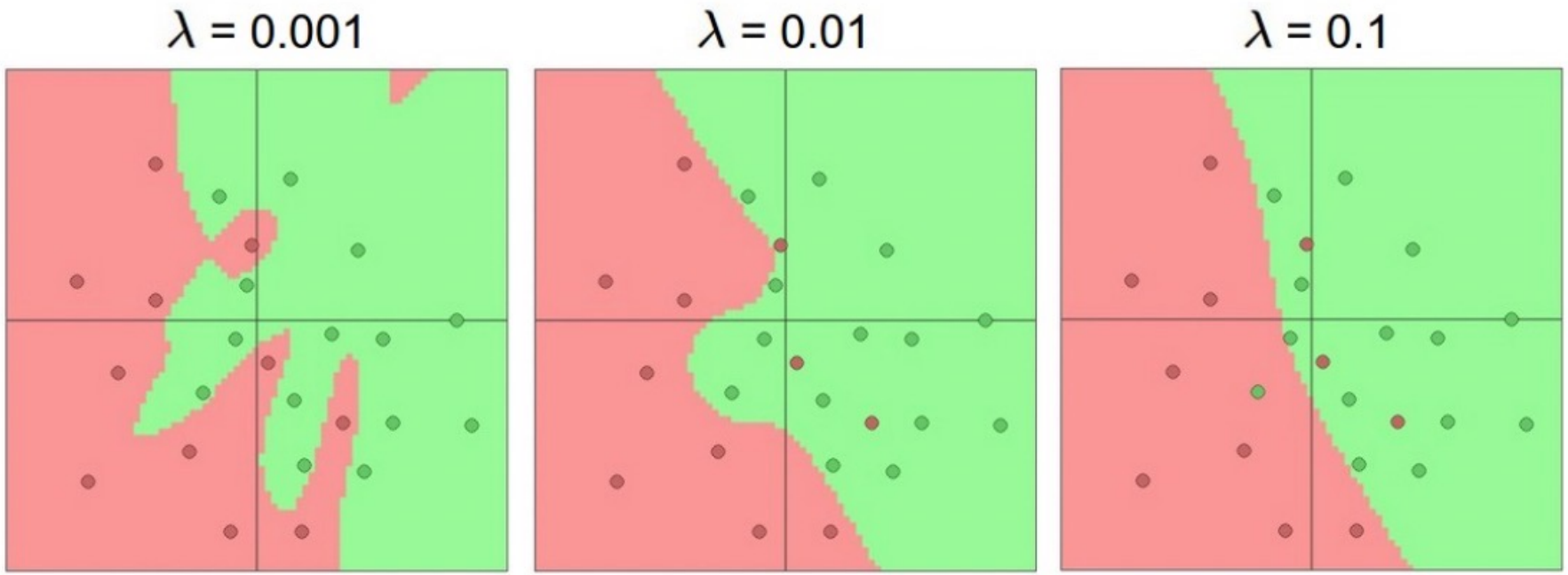
Points are linearly separable in features space!

Setting the number of layers and their sizes



More hidden units = more capacity

Don’t regularize with size; instead use stronger L2



(Web demo with ConvNetJS:
http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html)

Universal Approximation

Universal approximation theorem—Let $C(X, \mathbb{R}^m)$ denote the set of continuous functions from a subset X of a Euclidean $\mathbb{R}^n$ space to a Euclidean space $\mathbb{R}^m$. Let $\sigma \in C(\mathbb{R}, \mathbb{R})$. Note that $(\sigma \circ x)_i = \sigma(x_i)$, so $\sigma \circ x$ denotes σ applied to each component of x.

Then σ is not polynomial if and only if for every $n \in \mathbb{N}$, $m \in \mathbb{N}$, compact $K \subseteq \mathbb{R}^n$, $f \in C(K, \mathbb{R}^m), \varepsilon > 0$ there exist $k \in \mathbb{N}$, $A \in \mathbb{R}^{k \times n}$, $b \in \mathbb{R}^k$, $C \in \mathbb{R}^{m \times k}$ such that

$$\sup_{x \in K} \|f(x) - g(x)\| < \varepsilon$$

where $g(x) = C \cdot (\sigma \circ (A \cdot x + b))$

参考文献：HORNIK K, STINCHCOMBE M, WHITE H. Multilayer feedforward networks are universal approximators[J]. Neural networks, 1989, 2(5): 359-366.

Deep neural networks or **Wide** neural networks?

Theorem 5. *The maximal number of linear regions of the functions computed by a neural network with n_0 input units and L hidden layers, with $n_i \geq n_0$ rectifiers at the i-th layer, is lower bounded by*

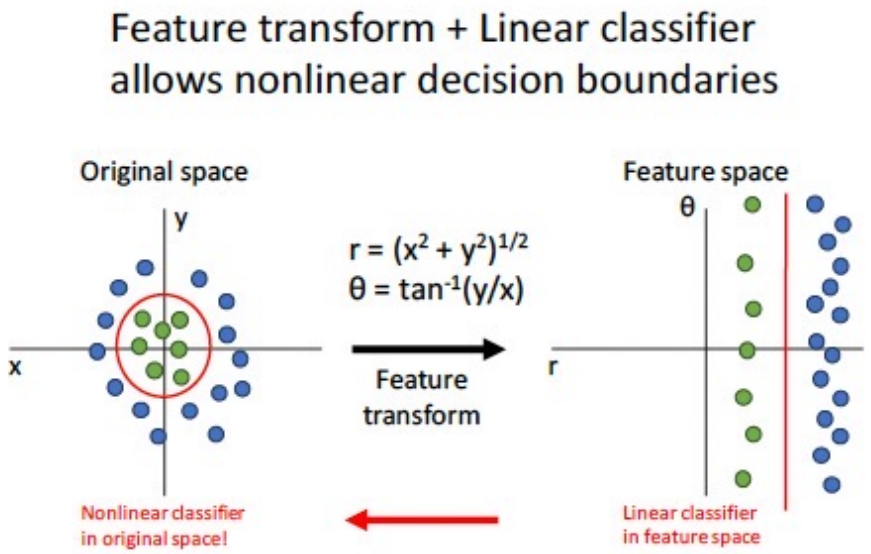
$$\left(\prod_{i=1}^{L-1} \left\lfloor \frac{n_i}{n_0} \right\rfloor^{n_0}\right) \sum_{j=0}^{n_0} \binom{n_L}{j}.$$

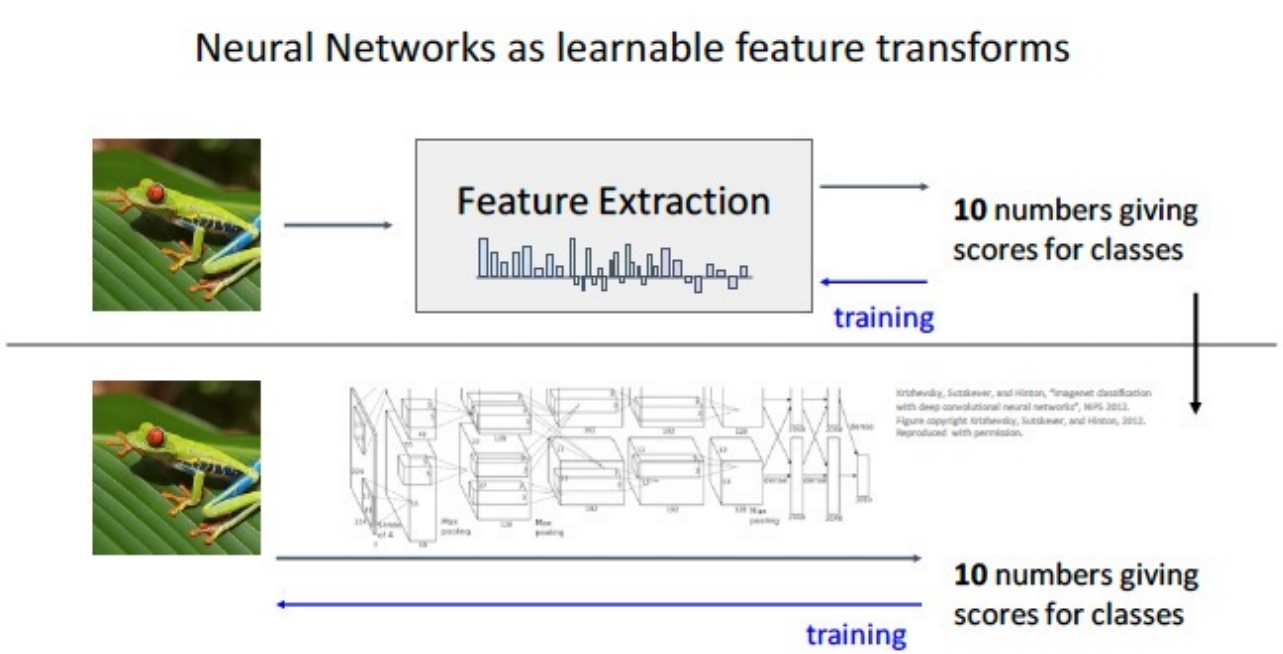
The next corollary gives an expression for the asymptotic behavior of these bounds. Assuming that $n_0 = O(1)$ and $n_i = n$ for all $i \geq 1$, the number of regions of a single layer model with Ln hidden units behaves as $O(L^{n_0} n^{n_0})$ (see Pascanu et al. 2013; Proposition 10). For a deep model, Theorem 5 implies:

Corollary 6. *A rectifier neural network with n_0 input units and L hidden layers of width $n \geq n_0$ can compute functions that have $\Omega\left(\left(n/n_0\right)^{(L-1)n_0} n^{n_0}\right)$ linear regions.*

参考文献：MONTUFAR G F, PASCANU R, CHO K, et al. On the number of linear regions of deep neural networks[J]. Advances in Neural Information Processing Systems, 2014, 27.

Summary

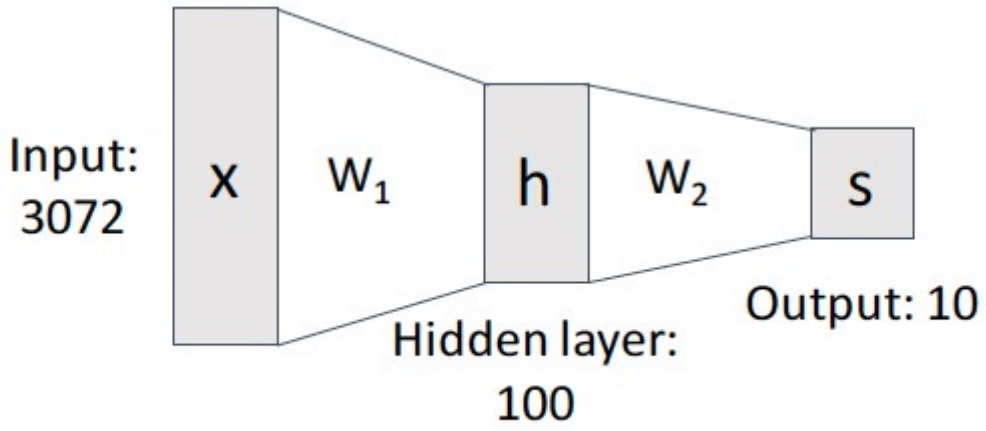

Feature transform + Linear classifier
allows nonlinear decision boundaries
Original space
y
x
$r = (x^2 + y^2)^{1/2}$
$\theta = \tan^{-1}(y/x)$
Feature
transform
Feature space
θ
r
Nonlinear classifier
in original space!
Linear classifier
in feature space


Neural Networks as learnable feature transforms
Feature Extraction
10 numbers giving
scores for classes
training
10 numbers giving
scores for classes
training

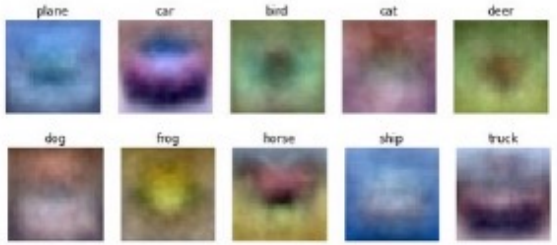
Summary

From linear classifiers to fully-connected networks

$$f = W_2 \max(0, W_1 x)$$



Linear classifier: One template per class



Neural networks: Many reusable templates



Next time

Backpropagation and Deep Neural Networks